

LION SOLVER

Sistema de geolocalización de oficinas universitarias con asistente conversacional (chatbot RAG)

Documentación técnica y de gestión del proyecto

Visión · Requerimientos · Scrum · Arquitectura · UML · Base de datos · Pruebas · Seguridad

Versión del documento: **1.0**

Versión del software: **4.1**

Autor del proyecto: **Kevin Ponce**

Fecha: **9 de octubre de 2026**

Contenido

Introducción 5
Control del documento 5
¿Qué es Lion Solver? 5
Arquitectura en un vistazo 6
Stack tecnológico 6
Estructura del repositorio 6
Mapa de la documentación 7
Guía de lectura por perfil 7
Convenciones 7
Nota sobre las fuentes 7

1. Visión del producto 9
1. Declaración de visión 9
2. Problema 9
3. Objetivos del producto 9
4. Alcance 9
5. Stakeholders y usuarios 10
6. Personas 10
7. Supuestos y restricciones 10
8. Glosario 11

2. Especificación de requerimientos 12
1. Actores 12
2. Requerimientos funcionales 12
3. Requerimientos no funcionales 14
4. Reglas de negocio 15
5. Matriz de trazabilidad (resumen) 15

3. Gestión ágil con Scrum 17
1. Equipo Scrum 17
2. Eventos 17
3. Artefactos y compromisos 18
4. Épicas 19
5. Mapa de historias (User Story Map) 20
6. Product Backlog 20
7. Sprints (reconstrucción histórica) 23
8. Roadmap propuesto 24
9. Tablero Kanban sugerido (GitHub Projects) 25

4. Arquitectura del sistema 26
1. Estilo arquitectónico 26
2. Modelo C4 26

3. Vista de despliegue....28
4. Flujos principales....29
5. Decisiones de arquitectura (ADR)....29
6. Atributos de calidad y tácticas....30

5. Diagramas UML....31

1. Diagrama de casos de uso....31
2. Diagrama de clases....33
3. Diagrama de paquetes....35
4. Diagramas de secuencia....36
5. Diagramas de actividad....41
6. Diagramas de estados....44
7. Diagrama de componentes....46
8. Diagrama de despliegue....47

6. Base de datos....48

1. Modelo entidad-relación....48
2. Modelo relacional....48
3. Diccionario de datos....49
4. Consultas principales....50
5. Consideraciones de integridad....50
6. Respaldo....51

7. Referencia de rutas HTTP....52

1. Resumen....52
2. Detalle....53
3. Manejo de errores....55
4. Mapa de navegación....55

8. Chatbot RAG....56

1. Visión general....56
2. Parámetros....57
3. Fase de ingesta....57
4. Fase de consulta....57
5. Costos y desempeño....58
6. Evaluación recomendada....58
7. Mejoras propuestas....59

9. Instalación y despliegue....60

1. Requisitos....60
2. Instalación local....60
3. Configuración....60
4. Ejecución....61
5. Despliegue en producción....62
6. Solución de problemas....63

10. Manual de usuario..... 64
1. Acceso al sistema.....64
2. Estudiantes.....64
3. Coordinadores de oficina.....65
4. Super administradores.....65
5. Preguntas frecuentes.....67

11. Plan de pruebas.....68
1. Alcance.....68
2. Estrategia.....68
3. Entorno.....69
4. Casos de prueba.....69
5. Ejemplos de pruebas automatizadas.....71
6. Criterios de entrada y salida.....72
7. Clasificación de defectos.....72

12. Seguridad, riesgos y deuda técnica..... 73
1. Controles existentes.....73
2. Hallazgos de seguridad.....73
3. Matriz de riesgos del proyecto.....75
4. Deuda técnica.....76
5. Plan de remediación.....78

Anexo A. Historial de cambios..... 79
Versión sin publicar.....79
Versión 4.1 - 2024-12-12.....79
Versión 4.0 - 2024-11-21.....79
Versión 3.0 - 2024-11-21.....79
Versión 2.0 - 2024-10-07.....80
Versión 1.2 - 2024-10-07.....80
Versión 1.1 - 2024-10-06.....80
Versión 1.0 - 2024-10-06.....80

Anexo B. Guía de contribución..... 81
Flujo de trabajo.....81
Estándares de código.....81
Pruebas.....82
Documentación.....82

Anexo C. Script de base de datos..... 83

Introducción

Control del documento

Campo	Valor
Proyecto	Lion Solver
Documento	Documentación técnica y de gestión del proyecto
Versión del documento	1.0
Versión del software	4.1 (commit c23db33, 12/12/2024)
Fecha	9 de octubre de 2026
Autor del proyecto	Kevin Ponce
Repositorio	https://github.com/KevinPonce90/LionSolver
Clasificación	Uso interno

¿Qué es Lion Solver?

Lion Solver es una aplicación web que ayuda a los estudiantes de un centro universitario (caso piloto: **CUCEI**) a **encontrar la oficina correcta para resolver un trámite** y a **llegar físicamente a ella**.

El estudiante escribe su duda en lenguaje natural (por ejemplo, *"¿dónde entrego mis papeles del servicio social?"*); el chatbot responde usando la información oficial que cada oficina ha cargado en PDF y **resalta en el mapa del campus** la oficina que corresponde.

Para el estudiante	Para el coordinador de oficina	Para el super administrador
Mapa interactivo del campus con todas las oficinas	Editar nombre, descripción, teléfono y horario de su oficina	Alta, edición y baja de **campus** (con polígono GeoJSON)
Chatbot que responde dudas de trámites	**Entrenar el chatbot** subiendo el PDF de su oficina	Alta, edición y baja de **oficinas** (con coordenadas)
Resaltado automático de la oficina en el mapa	Ver el mapa de su campus	Alta, edición y baja de **administradores**

Arquitectura en un vistazo

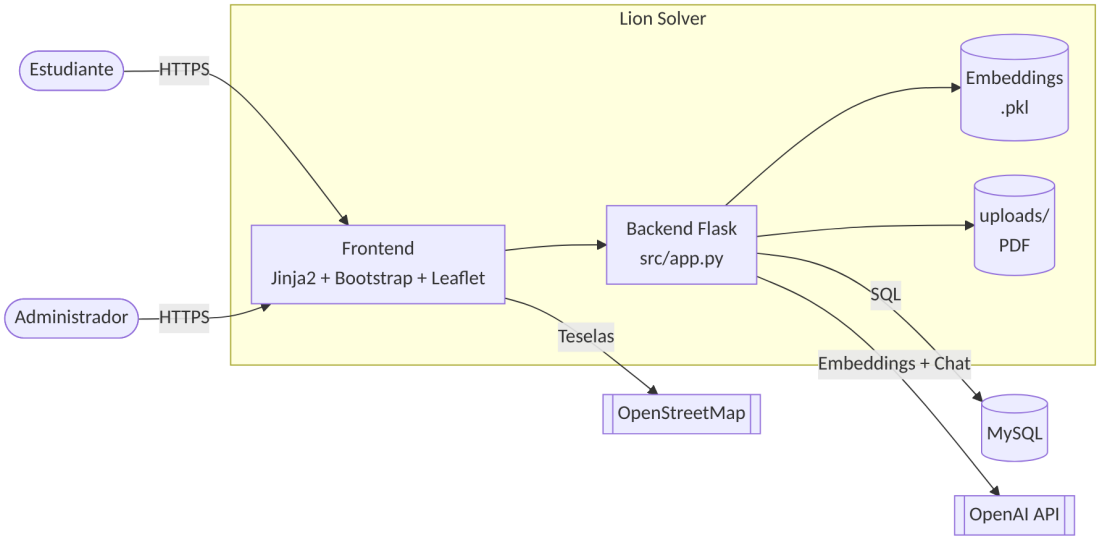


Figura 1. Arquitectura en un vistazo

Stack tecnológico

Capa	Tecnología
Backend	Python 3.12, Flask 3.0, Flask-Login, Flask-WTF (CSRF), Flask-MySQLdb
Base de datos	MySQL / MariaDB
IA	OpenAI `text-embedding-ada-002` (embeddings) y `gpt-3.5-turbo` (respuesta), `tiktoken`, `scikit-learn` (similitud coseno)
Procesamiento de documentos	PyPDF2
Frontend	Jinja2, Bootstrap 5.3, Leaflet, OpenStreetMap, JavaScript vanilla

Estructura del repositorio

```
LionSolver/
├── src/
│   ├── app.py                # Aplicación Flask (rutas, chatbot RAG, CRUD)
│   ├── app2.py               # Versión anterior sin chatbot (legado)
│   ├── config.py             # Configuración local (NO versionado)
│   ├── models/
│   │   ├── ModelUser.py      # Acceso a datos de estudiantes
│   │   ├── ModelAdmin.py     # Acceso a datos de administradores
│   │   └── entities/         # Entidades User y Admin (Flask-Login)
│   ├── template/             # Vistas Jinja2
│   └── static/               # CSS, JS (mapa, chatbot), imágenes, GeoJSON
├── embeddings/               # Índice vectorial por oficina
(embeddings_<office_id>.pkl)
├── uploads/                  # PDFs cargados por las oficinas
├── docs/                     # Documentación del proyecto
└── requirements.txt          # Dependencias (UTF-16)
```

Mapa de la documentación

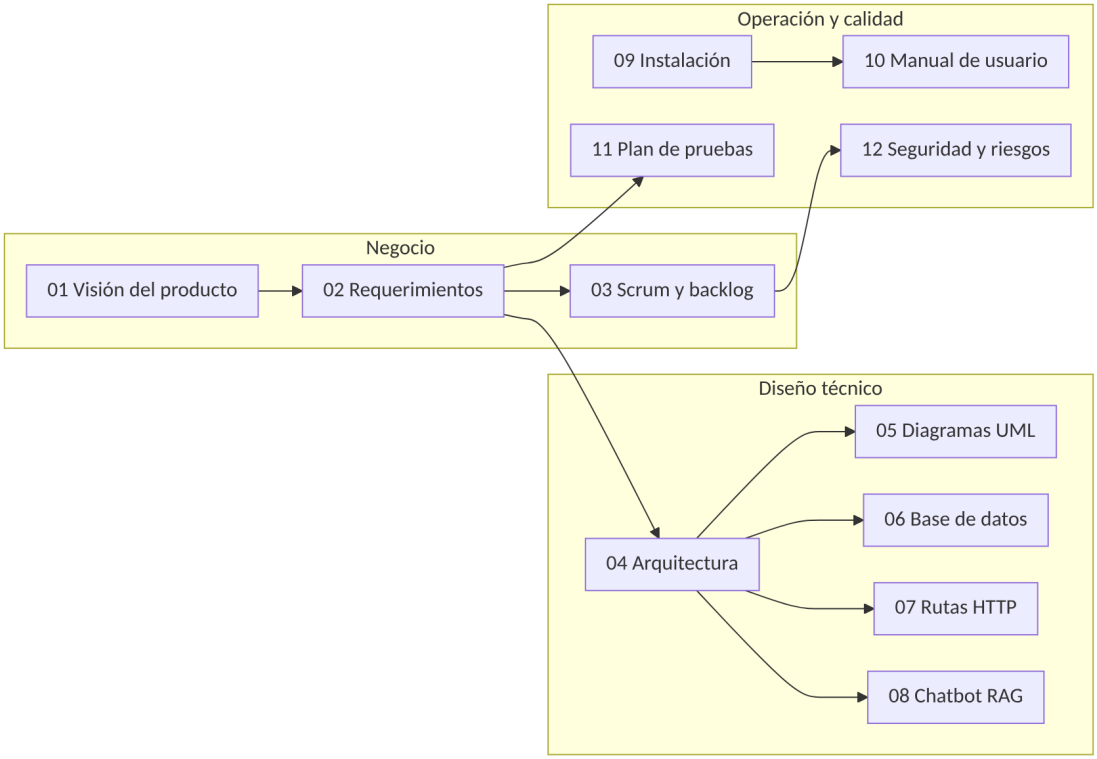


Figura 2. Mapa de la documentación

Guía de lectura por perfil

Perfil	Lectura recomendada
Product Owner / stakeholder	Capítulos 1 → 2 → 3 → 10
Desarrollador que se incorpora	Capítulos 9 → 4 → 5 → 6 → 7 → 8 → 12
QA / tester	Capítulos 2 → 11 → 7 → 10
Responsable de seguridad / operación	Capítulos 12 → 9 → 4

Convenciones

- **RF-xx / RNF-xx / RN-xx**: requerimiento funcional, no funcional y regla de negocio.
- **HU-xx**: historia de usuario del Product Backlog; **EP-xx**: épica.
- **CP-xx**: caso de prueba; **R-xx**: riesgo; **SEC-xx**: hallazgo de seguridad; **DT-xx**: deuda técnica.
- Las referencias a código usan la forma `archivo:línea`, por ejemplo `src/app.py:564`.

Nota sobre las fuentes

Esta documentación se elaboró a partir del código fuente y del historial de Git. Dos artefactos no están en el repositorio y se **reconstruyeron por inferencia**:

- **Esquema de base de datos**: deducido de las consultas SQL en `src/app.py` y `src/models/`. Ver capítulo 6 (Base de datos).
- **Planificación Scrum**: los sprints se reconstruyeron a partir de las fechas de los commits (versiones 1.0 a 4.1); las estimaciones en puntos de historia son de referencia. Ver capítulo 3 (Gestión ágil con Scrum).

1. Visión del producto

1. Declaración de visión

> **Para** los estudiantes de un centro universitario **que** no saben a qué oficina acudir para resolver un trámite ni dónde se encuentra físicamente, **Lion Solver** es una aplicación web de orientación **que** responde sus dudas en lenguaje natural y les señala en el mapa del campus la oficina correcta. **A diferencia de** los directorios estáticos, páginas web dispersas y la consulta presencial de oficina en oficina, **nuestro producto** concentra la información oficial de cada dependencia —cargada y mantenida por la propia oficina— y la combina con geolocalización.

2. Problema

Aspecto	Descripción
Problema	Los estudiantes desconocen qué oficina atiende cada trámite (servicio social, titulación, becas, coordinaciones de carrera…) y dónde está dentro de un campus extenso.
Afecta a	Estudiantes (especialmente de nuevo ingreso), personal de las oficinas que recibe consultas mal dirigidas.
Impacto	Tiempo perdido recorriendo el campus, filas en oficinas equivocadas, trámites fuera de plazo, carga de trabajo repetitiva para el personal.
Solución exitosa	Un único punto de consulta que responde *qué* hacer, *dónde* hacerlo y *cómo contactar* a la oficina.

3. Objetivos del producto

ID	Objetivo	Indicador de éxito (KPI propuesto)
OBJ-1	Reducir el tiempo que tarda un estudiante en identificar la oficina correcta	≥ 80 % de las preguntas de prueba se asocian a la oficina correcta
OBJ-2	Ubicar físicamente cada oficina en el campus	100 % de las oficinas registradas tienen coordenadas válidas dentro del polígono del campus
OBJ-3	Que cada oficina mantenga su propia información sin depender de TI	Una oficina puede actualizar datos y PDF en < 5 minutos
OBJ-4	Escalar a varios campus	Alta de un campus nuevo solo con datos administrativos (GeoJSON + oficinas), sin cambios de código

4. Alcance

4.1 Dentro del alcance (versión 4.1)

- Autenticación con código y contraseña para estudiantes y administradores.
- Mapa interactivo del campus del usuario (polígono GeoJSON + marcadores de oficinas).
- Chatbot con recuperación aumentada (RAG) sobre los PDF cargados por las oficinas.
- Resaltado automático en el mapa de la oficina asociada a la respuesta.
- Panel de coordinador de oficina: editar datos de contacto y cargar PDF.
- Panel de super administrador: CRUD de campus, oficinas y administradores.

4.2 Fuera del alcance

- Registro autónomo de estudiantes (se asume alta directa en la base de datos o integración futura con el sistema escolar).
- Navegación paso a paso (rutas a pie) dentro del campus.
- Historial de conversaciones del chatbot.
- Aplicación móvil nativa.
- Integración con sistemas de trámites o citas.

5. Stakeholders y usuarios

Stakeholder	Rol en el sistema	Interés principal	Nivel de influencia
Estudiante	`User`	Resolver dudas y encontrar oficinas rápido	Alto (usuario final)
Coordinador / responsable de oficina	`Admin` regular (`admin_priv = 0`)	Publicar información correcta de su oficina	Medio
Super administrador	`Admin` super (`admin_priv = 1`)	Mantener el catálogo de campus, oficinas y cuentas	Alto
Dirección del centro universitario	— (patrocinador)	Mejorar la atención estudiantil	Alto
Equipo de desarrollo	—	Mantenibilidad, costos de API	Medio

6. Personas

Ana, 18 años — estudiante de nuevo ingreso. Usa el celular para todo. No conoce el campus y no sabe que "Servicio Social" tiene una unidad propia. Quiere una respuesta en una frase y ver en el mapa a dónde caminar.

Luis, 42 años — coordinador de la Unidad de Servicio Social. Recibe todos los días las mismas preguntas. Tiene sus lineamientos en un PDF. Quiere subir ese PDF una vez y que el sistema conteste por él.

Marta, 35 años — administradora de sistemas del centro. Da de alta campus, oficinas y coordinadores. Necesita formularios simples y control sobre quién puede editar qué.

7. Supuestos y restricciones

Tipo	Descripción
Supuesto	Los estudiantes ya existen en la tabla `users` con su campus asignado.
Supuesto	Cada administrador está asociado a exactamente una oficina y un campus.
Supuesto	Los PDF de las oficinas contienen texto extraíble (no son imágenes escaneadas).
Restricción	Dependencia de un servicio externo de pago (OpenAI) para embeddings y respuestas.
Restricción	El mapa base depende de las teselas públicas de OpenStreetMap.
Restricción	El índice vectorial se mantiene en memoria del proceso (un solo proceso de servidor).

8. Glosario

Término	Definición
Campus	Centro universitario con un polígono geográfico (GeoJSON) y un punto central (lat/lon). Su clave es el nombre.
Oficina	Dependencia física dentro de un campus, con coordenadas, carrera/área, contacto y horario.
Código	Identificador numérico institucional usado como usuario para iniciar sesión.
Coordinador	Administrador regular responsable de una oficina.
Super administrador	Administrador con `admin_priv = 1`; gestiona campus, oficinas y cuentas.
Embedding	Vector numérico (1 536 dimensiones con `text-embedding-ada-002`) que representa el significado de un texto.
Fragmento (chunk)	Porción del texto de un PDF de máximo 500 tokens.
RAG	*Retrieval-Augmented Generation*: se recuperan fragmentos relevantes y se pasan como contexto al modelo generativo.
GeoJSON	Formato estándar (RFC 7946) para representar geometrías geográficas.
Similitud coseno	Medida de cercanía entre dos vectores, usada para elegir los fragmentos más relevantes.

2. Especificación de requerimientos

Formato inspirado en IEEE 830 / ISO/IEC/IEEE 29148. Prioridad según **MoSCoW** (*Must*, *Should*, *Could*, *Won't*). La columna **Estado** refleja la versión 4.1.

Leyenda de estado: ✔️ implementado · ⚠️ implementado con observaciones (ver capítulo 12 (Seguridad, riesgos y deuda técnica)) · ○ pendiente.

1. Actores

Actor	Descripción	Hereda de
Visitante	Persona no autenticada	—
Usuario autenticado	Cualquier persona con sesión activa	Visitante
Estudiante	Usuario de la tabla `users` con `user_active = 1`	Usuario autenticado
Coordinador de oficina	Administrador con `admin_priv = 0`	Usuario autenticado
Super administrador	Administrador con `admin_priv = 1`	Coordinador de oficina
OpenAI API	Sistema externo que genera embeddings y respuestas	—

2. Requerimientos funcionales

2.1 Autenticación y sesión

ID	Requerimiento	Actor	Prioridad	Estado	Implementación
RF-01	El sistema debe permitir iniciar sesión con **código** y **contraseña**.	Visitante	Must	✔️	`POST /login`
RF-02	El sistema debe distinguir si el código pertenece a un estudiante o a un administrador y redirigir a `/home` o `/admin` respectivamente.	Sistema	Must	✔️	`src/app.py:222`
RF-03	El sistema debe rechazar el acceso de estudiantes inactivos (`user_active ≠ 1`).	Sistema	Must	✔️	`src/app.py:232`
RF-04	El sistema debe mostrar un mensaje cuando el usuario no exista o la contraseña sea incorrecta.	Sistema	Must	✔️	`flash()`
RF-05	El sistema debe permitir cerrar sesión.	Usuario autenticado	Must	✔️	`GET /logout`
RF-06	Las vistas privadas deben redirigir al login si no hay sesión.	Sistema	Must	✔️	`@login_required` + manejador 401

2.2 Mapa del campus

ID	Requerimiento	Actor	Prioridad	Estado	Implementación
RF-07	Mostrar un mapa interactivo centrado en el campus del usuario.	Usuario autenticado	Must	✔️	`GET /home`, `mapa.js`
RF-08	Dibujar el perímetro del campus a partir de su GeoJSON.	Sistema	Must	✔️	`L.geoJSON`
RF-09	Mostrar un marcador por oficina con ventana emergente (nombre y descripción).	Sistema	Must	✔️	`L.marker().bindPopup()`
RF-10	Resaltar, centrar y abrir la oficina indicada por el chatbot.	Sistema	Should	✔️	`highlightOffice()`

2.3 Chatbot

ID	Requerimiento	Actor	Prioridad	Estado	Implementación
RF-11	Permitir escribir preguntas en lenguaje natural desde una ventana flotante.	Usuario autenticado	Must	✔️	`chatbot.js`
RF-12	Responder usando únicamente el contenido de los PDF cargados por las oficinas como contexto.	Sistema	Must	✔️	`POST /chatbot`
RF-13	Indicar al final de la respuesta la oficina correspondiente y cómo contactarla.	Sistema	Should	⚠️	Prompt en `get_answer()`; `office_info` no se inyecta en el prompt
RF-14	Devolver el `office_id` asociado para resaltarlo en el mapa.	Sistema	Should	✔️	JSON `{reply, office_id}`
RF-15	Responder saludos simples sin consultar el modelo.	Sistema	Could	✔️	`is_greeting()`
RF-16	Informar si el chatbot aún no tiene conocimiento cargado.	Sistema	Should	✔️	`src/app.py:572`

2.4 Gestión de la oficina (coordinador)

ID	Requerimiento	Actor	Prioridad	Estado	Implementación
RF-17	Mostrar un panel con accesos a: ver mapa, editar oficina y entrenar chatbot.	Coordinador	Must	✔️	`admin.html`
RF-18	Editar nombre, descripción, teléfono y horario de la propia oficina.	Coordinador	Must	✔️	`POST /office_Edit`
RF-19	Subir un PDF que alimente la base de conocimiento de la oficina.	Coordinador	Must	✔️	`POST /upload_pdf`

ID	Requerimiento	Actor	Prioridad	Estado	Implementación
RF-20	Al subir un nuevo PDF, reemplazar el conocimiento anterior de esa oficina y recargar el índice sin reiniciar el servidor.	Sistema	Must	✔	`embeddings_<office_id>.pkl` + recarga

2.5 Administración general (super administrador)

ID	Requerimiento	Actor	Prioridad	Estado	Implementación
RF-21	Listar, crear, editar y eliminar **campus** (nombre, lat, lon, archivo GeoJSON).	Super admin	Must	✔	`/campus_view`, `/campus_Add`, `/campus_Edit`, `/campus_Delete`
RF-22	Listar, crear, editar y eliminar **oficinas** (campus, nombre, lat, lon, carrera).	Super admin	Must	✔	`/office_View`, `/office_Add`, `/office_Edit_Rec`, `/office_Delete`
RF-23	Listar, crear, editar y eliminar **administradores** (código, contraseña, nombre, correo, campus, oficina, privilegio).	Super admin	Must	✔	`/admin_View`, `/admin_Add`, `/admin_Edit`, `/admin_Delete`
RF-24	Mostrar en el listado de oficinas el coordinador asignado.	Super admin	Could	✔	`LEFT JOIN admins`
RF-25	Mostrar mensajes de confirmación tras cada operación.	Sistema	Should	✔	`flash()`
RF-26	Restringir las operaciones de administración según el rol.	Sistema	Must	○	Ver SEC-01
RF-27	Administrar cuentas de estudiantes desde la interfaz.	Super admin	Could	○	No implementado

3. Requerimientos no funcionales

ID	Categoría (ISO 25010)	Requerimiento	Métrica / criterio	Estado
RNF-01	Seguridad	Las contraseñas se almacenan con hash (Werkzeug).	Nunca en texto plano	✔
RNF-02	Seguridad	Todos los formularios y la llamada al chatbot llevan token CSRF.	Flask-WTF `CSRFProtect`	✔ (solo al ejecutar con `python src/app.py`)
RNF-03	Seguridad	Las consultas SQL son parametrizadas.	Sin concatenación de entradas	✔
RNF-04	Seguridad	Los secretos (API key, credenciales BD) no se versionan.	`.env` y `config.py` en `.gitignore`	✔
RNF-05	Seguridad	Control de acceso basado en roles en todas las rutas de administración.	403 para roles no autorizados	○

ID	Categoría (ISO 25010)	Requerimiento	Métrica / criterio	Estado
RNF-06	Desempeño	Respuesta del chatbot en tiempo aceptable.	p95 < 8 s (dominado por la API externa)	Por medir
RNF-07	Desempeño	Carga del mapa.	< 3 s en red 4G	Por medir
RNF-08	Usabilidad	Interfaz en español y adaptable a móvil.	Bootstrap responsive, `viewport`	✔️
RNF-09	Usabilidad	El chatbot no debe tapar el mapa de forma permanente.	Ventana minimizable	✔️
RNF-10	Mantenibilidad	Separación entidad / acceso a datos.	Paquete `models/`	⚠️ (la mayoría del SQL está en `app.py`)
RNF-11	Portabilidad	Ejecutable en Windows y Linux con Python 3.12.	`requirements.txt`	✔️
RNF-12	Escalabilidad	Soportar varios campus sin cambios de código.	Datos en tabla `campus`	✔️
RNF-13	Disponibilidad	Degradación controlada si OpenAI falla.	Mensaje de error amable	✔️
RNF-14	Costo	Minimizar llamadas a la API.	1 embedding + 1 completion por pregunta	✔️

4. Reglas de negocio

ID	Regla
RN-01	Un estudiante pertenece a un único campus y solo ve el mapa de ese campus.
RN-02	Un administrador pertenece a un campus y a una oficina.
RN-03	`admin_priv = 1` ⇒ super administrador; `admin_priv = 0` ⇒ coordinador de oficina.
RN-04	Solo pueden iniciar sesión los estudiantes con `user_active = 1`.
RN-05	Cada oficina tiene como máximo un archivo de conocimiento activo (`embeddings_<office_id>.pkl`); subir un PDF nuevo reemplaza al anterior.
RN-06	El campus se identifica por su nombre; el nombre no es editable una vez creado.
RN-07	El polígono del campus debe ser un archivo GeoJSON válido.
RN-08	La respuesta del chatbot se asocia a la oficina del fragmento con mayor similitud.
RN-09	Los códigos de estudiante y de administrador comparten espacio de identificadores; si coinciden, prevalece el estudiante al recargar la sesión (ver SEC-05).

5. Matriz de trazabilidad (resumen)

Requerimiento	Historia de usuario	Caso de prueba
RF-01 a RF-06	HU-01, HU-02, HU-03	CP-01 a CP-06
RF-07 a RF-10	HU-04, HU-05	CP-07 a CP-10
RF-11 a RF-16	HU-06, HU-07, HU-08	CP-11 a CP-17

Requerimiento	Historia de usuario	Caso de prueba
RF-17 a RF-20	HU-09, HU-10	CP-18 a CP-22
RF-21	HU-11	CP-23 a CP-26
RF-22	HU-12	CP-27 a CP-29
RF-23	HU-13	CP-30 a CP-32
RF-26 / RNF-05	HU-14	CP-33, CP-34

3. Gestión ágil con Scrum

> **Nota metodológica.** El historial del repositorio muestra entregas incrementales (versiones 1.0 → 4.1, entre octubre y diciembre de 2024). Los sprints de esta sección se **reconstruyeron a partir de esas entregas** para dejar un marco Scrum formal sobre el que continuar el proyecto. Las estimaciones en *story points* son de referencia (escala Fibonacci) y deben recalibrarse en el siguiente *Sprint Planning*.

1. Equipo Scrum

Rol	Responsable	Responsabilidades
Product Owner	Por asignar (representante del centro universitario)	Maximizar el valor del producto, ordenar el Product Backlog, aceptar incrementos
Scrum Master	Por asignar	Facilitar eventos, eliminar impedimentos, proteger al equipo
Developers	Kevin Ponce (@KevinPonce90)	Construir el incremento: backend, frontend, IA, base de datos

Stakeholders: estudiantes, coordinadores de oficina, dirección del centro.

2. Eventos

Evento	Duración (sprint de 2 semanas)	Entradas	Salidas
Sprint Planning	2 h	Product Backlog ordenado, capacidad, Definition of Done	Sprint Goal, Sprint Backlog
Daily Scrum	15 min	Avance del día anterior	Plan para las próximas 24 h, impedimentos
Sprint Review	1 h	Incremento terminado	Feedback, backlog actualizado
Sprint Retrospective	45 min	Datos del sprint	Acciones de mejora (máx. 3)
Backlog Refinement	≤ 10 % de la capacidad	Historias candidatas	Historias que cumplen la DoR

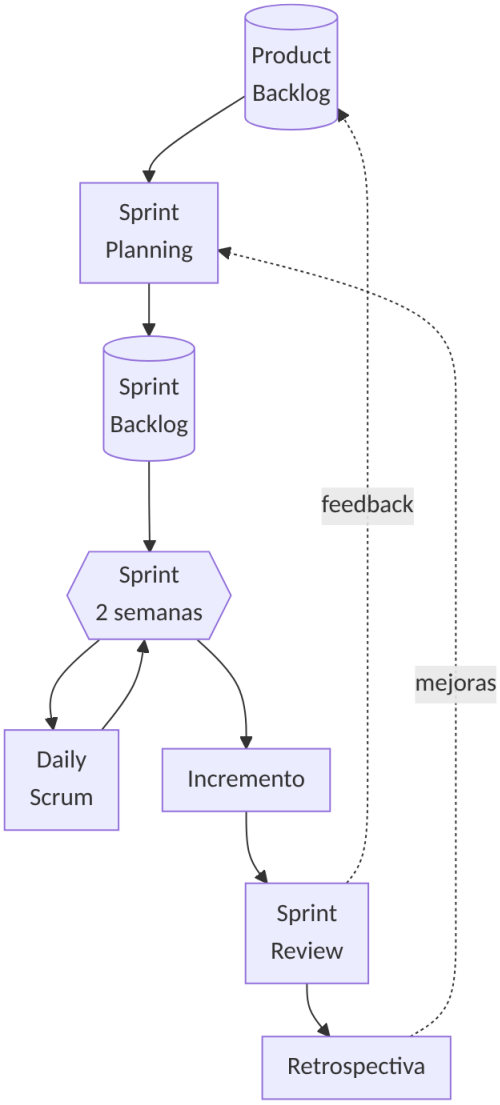


Figura 3. Ciclo de eventos de Scrum

3. Artefactos y compromisos

Artefacto	Compromiso	Dónde vive
Product Backlog	Objetivo del producto: *"Que cualquier estudiante encuentre la oficina correcta y llegue a ella en menos de 2 minutos."*	Sección 6 de este documento / GitHub Issues
Sprint Backlog	Sprint Goal	GitHub Projects (tablero)
Incremento	Definition of Done	Rama `main` + etiqueta de versión

3.1 Definition of Ready (DoR)

Una historia entra a un sprint cuando:

- ☐ Sigue el formato *Como… quiero… para…* y aporta valor identificable.
- ☐ Tiene criterios de aceptación verificables (Gherkin o lista).

- ☐ Está estimada por el equipo.
- ☐ Sus dependencias (datos, diseños, credenciales) están resueltas.
- ☐ Cumple INVEST (Independiente, Negociable, Valiosa, Estimable, Pequeña, *Testable*).

3.2 Definition of Done (DoD)

Un incremento está terminado cuando:

- ☐ El código está en `main` mediante *pull request* revisado.
- ☐ Cumple todos los criterios de aceptación.
- ☐ Tiene pruebas automatizadas para la lógica nueva (cuando aplique) y pasa la suite completa.
- ☐ Las rutas nuevas tienen `@login_required` y verificación de rol cuando corresponda.
- ☐ No introduce secretos en el repositorio.
- ☐ La documentación (`docs/`) y el `CHANGELOG.md` están actualizados.
- ☐ Fue demostrado en la Sprint Review y aceptado por el Product Owner.

4. Épicas

ID	Épica	Objetivo	Versión
EP-01	Autenticación y control de acceso	Que cada persona entre con su código y vea solo lo que le corresponde	1.1
EP-02	Mapa interactivo del campus	Ubicar visualmente todas las oficinas	1.1 → 3.0
EP-03	Chatbot RAG	Responder dudas con información oficial	1.1 → 4.0
EP-04	Autogestión de oficinas	Que cada coordinador mantenga su información	1.1 → 1.2
EP-05	Administración del sistema	Gestionar campus, oficinas y cuentas	1.1 → 1.2
EP-06	Calidad, seguridad y operación	Hacer el sistema confiable y desplegable	Backlog

5. Mapa de historias (User Story Map)

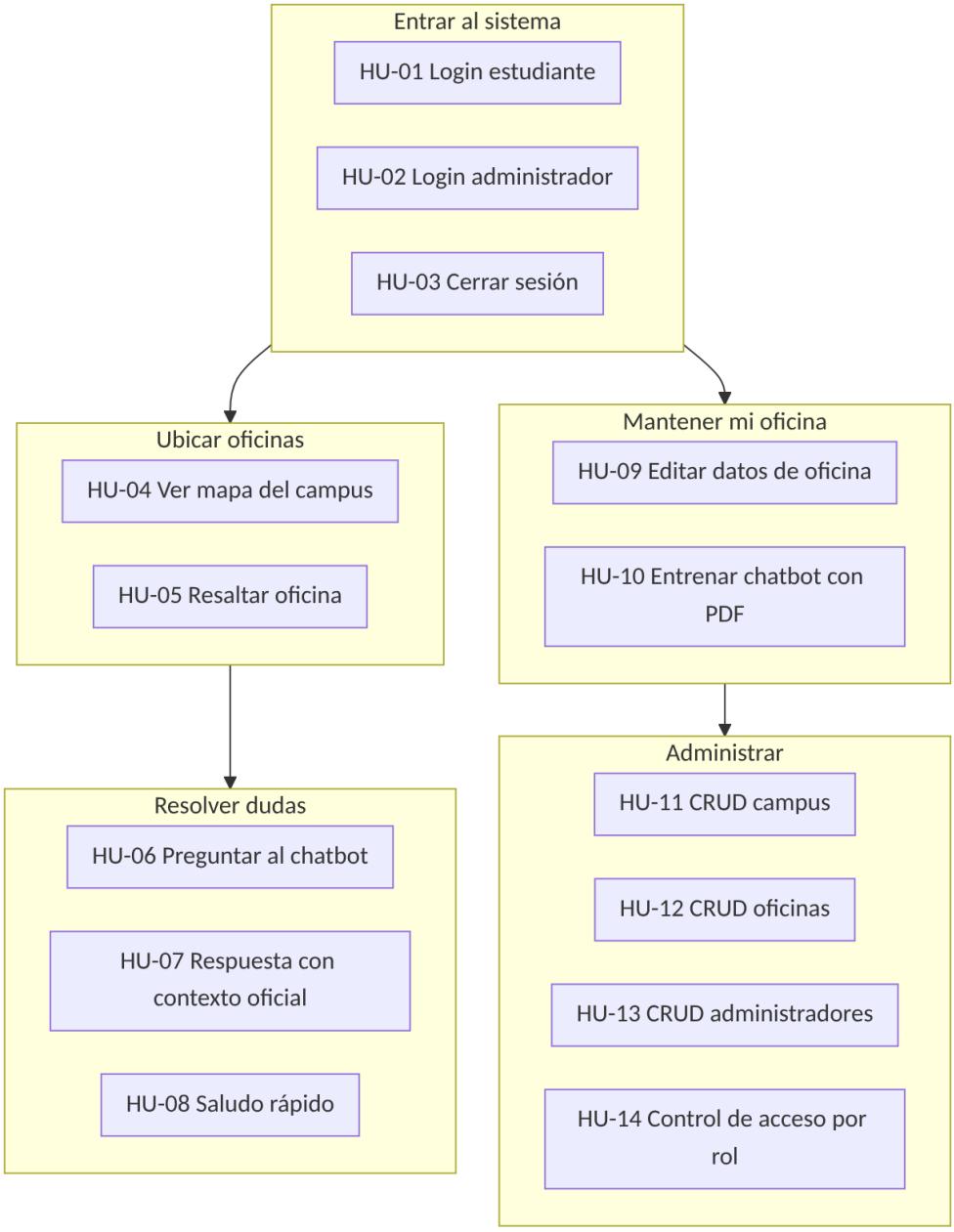


Figura 4. Mapa de historias (User Story Map)

6. Product Backlog

6.1 Historias entregadas

ID	Épica	Historia de usuario	SP	Prioridad	Sprint	Estado
HU-01	EP-01	**Como** estudiante **quiero** iniciar sesión con mi código y contraseña **para** acceder al mapa de mi campus.	3	Must	1	Hecho

ID	Épica	Historia de usuario	SP	Prioridad	Sprint	Estado
HU-02	EP-01	**Como** administrador **quiero** iniciar sesión con mi código **para** acceder a mi panel.	2	Must	1	Hecho
HU-03	EP-01	**Como** usuario **quiero** cerrar sesión **para** proteger mi cuenta en equipos compartidos.	1	Must	1	Hecho
HU-04	EP-02	**Como** estudiante **quiero** ver el mapa de mi campus con todas las oficinas **para** saber dónde está cada una.	5	Must	1 → 3	Hecho
HU-05	EP-02	**Como** estudiante **quiero** que el mapa resalte la oficina que me sugiere el chatbot **para** no tener que buscarla.	5	Should	3	Hecho
HU-06	EP-03	**Como** estudiante **quiero** escribir mi duda en una ventana de chat **para** no tener que navegar por varias páginas.	3	Must	1	Hecho
HU-07	EP-03	**Como** estudiante **quiero** recibir respuestas basadas en la información oficial de las oficinas **para** confiar en ellas.	8	Must	1 → 2	Hecho
HU-08	EP-03	**Como** estudiante **quiero** que el bot responda a un saludo **para** que la conversación sea natural.	1	Could	1	Hecho
HU-09	EP-04	**Como** coordinador **quiero** editar nombre, descripción, teléfono y horario de mi oficina **para** mantenerla actualizada.	3	Must	1	Hecho
HU-10	EP-04	**Como** coordinador **quiero** subir el PDF de mi oficina **para** que el chatbot conteste las preguntas frecuentes.	8	Must	1 → 2	Hecho
HU-11	EP-05	**Como** super admin **quiero** gestionar campus con su polígono GeoJSON **para** dar soporte a varios centros.	5	Must	1	Hecho
HU-12	EP-05	**Como** super admin **quiero** gestionar oficinas con sus coordenadas **para** que aparezcan en el mapa.	5	Must	2	Hecho
HU-13	EP-05	**Como** super admin **quiero** gestionar cuentas de administradores **para** asignar coordinadores a oficinas.	5	Must	1	Hecho

6.2 Backlog pendiente (ordenado por prioridad)

ID	Épica	Historia de usuario	SP	Prioridad	Origen
HU-14	EP-01	**Como** super admin **quiero** que solo mi rol pueda ejecutar operaciones de administración **para** evitar cambios no autorizados.	5	Must	SEC-01
HU-15	EP-04	**Como** coordinador **quiero** que el formulario de edición muestre los datos actuales de mi oficina **para** no borrarlos por accidente.	2	Must	DT-06
HU-16	EP-03	**Como** estudiante **quiero** que la respuesta incluya el teléfono y horario reales de la oficina **para** contactarla.	3	Should	RF-13
HU-17	EP-06	**Como** equipo **queremos** migrar al SDK actual de OpenAI y a modelos vigentes **para** evitar la obsolescencia de la API.	5	Should	DT-01

ID	Épica	Historia de usuario	SP	Prioridad	Origen
HU-18	EP-06	**Como** equipo **queremos** pruebas automatizadas y CI **para** detectar regresiones.	8	Should	DT-08
HU-19	EP-03	**Como** estudiante **quiero** ver un indicador de “escribiendo…” **para** saber que el bot está procesando.	1	Could	UX
HU-20	EP-05	**Como** super admin **quiero** gestionar cuentas de estudiantes **para** no depender de la base de datos directa.	8	Could	RF-27
HU-21	EP-06	**Como** operador **quiero** desplegar con Docker y Gunicorn **para** tener un entorno de producción reproducible.	5	Should	DT-10
HU-22	EP-02	**Como** estudiante **quiero** ver mi ubicación actual en el mapa **para** orientarme hacia la oficina.	3	Could	Feedback
HU-23	EP-03	**Como** estudiante **quiero** que el bot responda “no lo sé” cuando la información no exista **para** no recibir datos inventados.	3	Should	Calidad IA

6.3 Criterios de aceptación (selección, formato Gherkin)

HU-01 · Login de estudiante

```
Escenario: Estudiante activo con credenciales correctas
  Dado que existe el estudiante con código "219000001" activo
  Cuando ingresa su código y su contraseña correcta
  Entonces es redirigido a "/home"
  Y ve el mapa de su campus

Escenario: Estudiante inactivo
  Dado que el estudiante "219000002" tiene user_active = 0
  Cuando ingresa credenciales correctas
  Entonces permanece en "/login"
  Y ve el mensaje "Contraseña incorrecta o Estudiante no activo"

Escenario: Código inexistente
  Cuando ingresa un código que no existe
  Entonces ve el mensaje "Usurario no encontrado"
```

HU-05 · Resaltar oficina

```
Escenario: El chatbot identifica una oficina
  Dado que estoy en "/home" con el mapa cargado
  Cuando el chatbot responde con office_id = 6
  Entonces el marcador de la oficina 6 cambia al icono resaltado
  Y el mapa se centra en ella con zoom 18
  Y se abre su ventana emergente
```

HU-10 · Entrenar chatbot con PDF

```
Escenario: PDF con texto
  Dado que soy coordinador de la oficina 6
  Cuando subo "lineamientos.pdf" con texto extraíble
  Entonces se crea o reemplaza "embeddings/embeddings_6.pkl"
  Y el índice en memoria se recarga sin reiniciar el servidor
  Y veo "El PDF ha sido procesado y los embeddings se han generado correctamente."

Escenario: PDF sin texto (escaneado)
  Cuando subo un PDF del que no se puede extraer texto
  Entonces no se generan embeddings
  Y se me informa que el PDF no contiene texto
```

HU-14 · Control de acceso por rol

```
Escenario: Estudiante intenta crear un administrador
  Dado que tengo sesión como estudiante
  Cuando envío POST a "/admin_Add"
  Entonces recibo HTTP 403
  Y no se crea ningún registro
```

7. Sprints (reconstrucción histórica)

Sprint	Periodo	Sprint Goal	Historias	SP	Release
Sprint 0	hasta 2024-10-06	Preparar repositorio y visión	Setup, README	—	v1.0
Sprint 1	→ 2024-10-06	*"Un usuario puede entrar, ver el mapa de su campus y hacer una pregunta al chatbot."*	HU-01, 02, 03, 04, 06, 07 (base), 08, 09, 10 (base), 11, 13	39	v1.1
Sprint 2	2024-10-07	*"El super admin gestiona oficinas y el chatbot procesa PDFs largos de forma robusta."*	HU-12, HU-07 (fragmentación por tokens), HU-10 (carga a `uploads/`)	21	v1.2, v2.0
Sprint 3	2024-10-08 → 2024-11-21	*"El chatbot señala en el mapa la oficina de la respuesta."*	HU-04 (migración Folium → Leaflet), HU-05	10	v3.0, v4.0
Sprint 4	2024-11-22 → 2024-12-12	*"Estabilizar la versión candidata."*	Limpieza de `chatbot.js`	1	v4.1

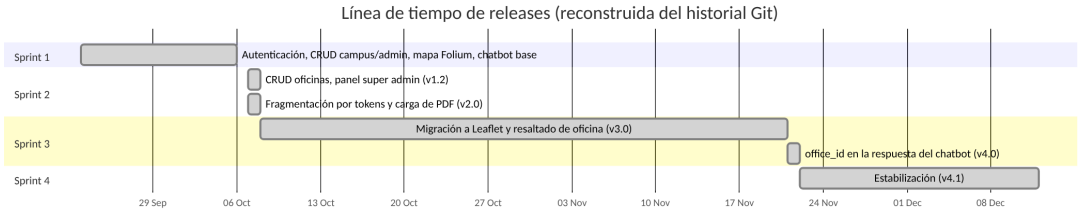


Figura 5. Línea de tiempo de releases (reconstruida del historial Git)

7.1 Velocidad

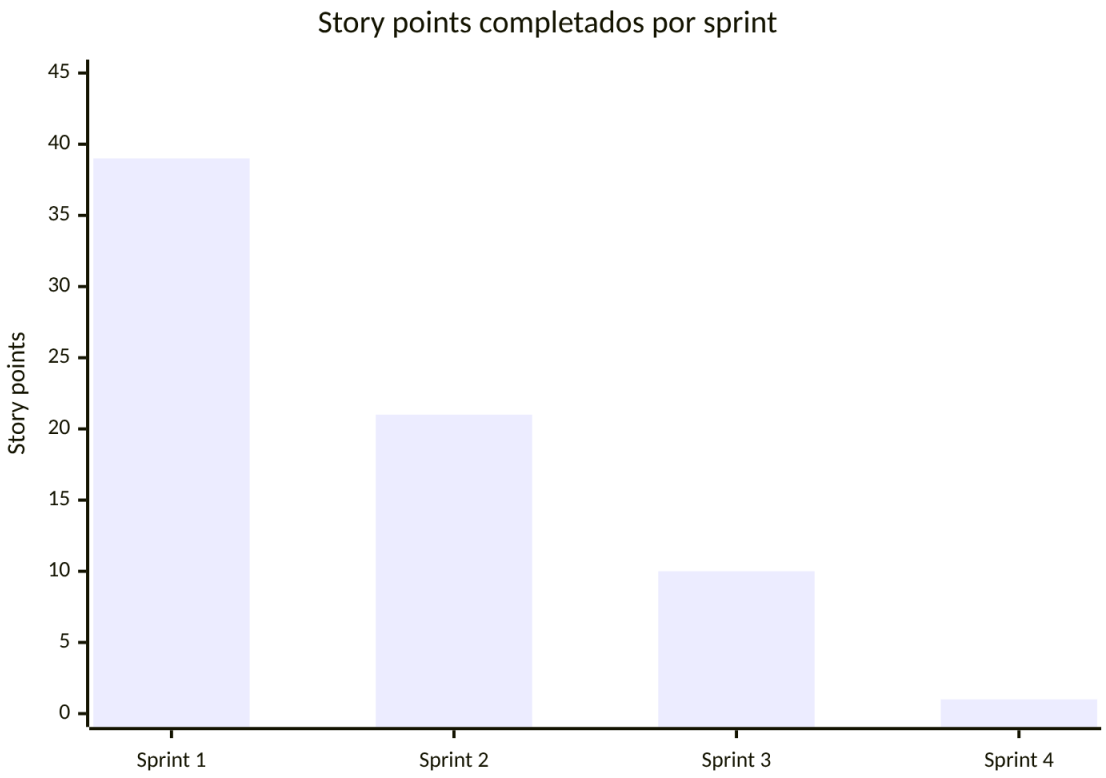


Figura 6. Story points completados por sprint

Velocidad media de referencia: **~18 SP/sprint**. El Sprint 1 concentra la mayor parte del alcance porque el primer commit funcional (v1.1) incluyó casi toda la aplicación; en adelante se recomienda planificar con una capacidad de **15–20 SP** por sprint de dos semanas.

7.2 Retrospectivas (síntesis)

Sprint	¿Qué salió bien?	¿Qué mejorar?	Acción
1	Se entregó un producto usable de extremo a extremo.	Commit único muy grande; sin pruebas.	Commits pequeños por historia; PR por historia.
2	La fragmentación por tokens resolvió PDFs largos.	`app2.py` duplicado queda como código muerto.	Eliminar código legado o moverlo a una rama.
3	Leaflet permitió interacción entre chatbot y mapa.	La métrica “oficina más común” se cambió por “top-1” sin medir precisión.	Crear un conjunto de preguntas de evaluación.
4	Versión estable para demo.	No hay control de acceso por rol en rutas admin.	Priorizar HU-14 al inicio del siguiente sprint.

8. Roadmap propuesto

Sprint	Sprint Goal propuesto	Historias	SP
Sprint 5	*“Solo quien debe puede administrar.”*	HU-14, HU-15, HU-16	10
Sprint 6	*“El chatbot es mantenible y honesto.”*	HU-17, HU-23, HU-19	9

Sprint	Sprint Goal propuesto	Historias	SP
Sprint 7	*"Cada cambio se prueba y se despliega igual."*	HU-18, HU-21	13
Sprint 8	*"Más autonomía para administradores y estudiantes."*	HU-20, HU-22	11

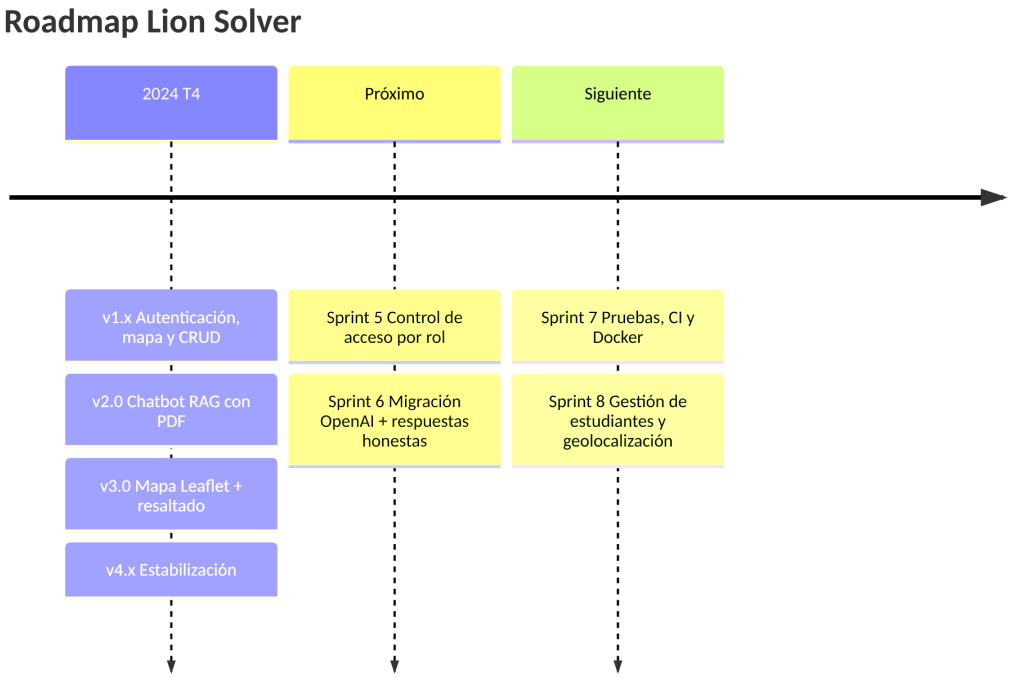


Figura 7. Roadmap del producto

9. Tablero Kanban sugerido (GitHub Projects)

Columna	Política (WIP)
Backlog	Historias refinadas que cumplen DoR
Sprint Backlog	Comprometidas en el Sprint Planning
En progreso	WIP ≤ 2 por developer
En revisión	PR abierto, esperando revisión
Hecho	Cumple la DoD

Etiquetas recomendadas: `épica:EP-0x`, `tipo:historia`, `tipo:bug`, `tipo:deuda-técnica`, `prioridad:must|should|could`, `sp:1|2|3|5|8|13`.

4. Arquitectura del sistema

1. Estilo arquitectónico

Lion Solver es un **monolito web de renderizado en servidor** (Flask + Jinja2) con un patrón **MVC simplificado**:

Capa MVC	Implementación
Modelo	`src/models/entities/` (entidades `User`, `Admin`) y `src/models/` (`ModelUser`, `ModelAdmin` — acceso a datos). El resto del SQL vive en las funciones de vista.
Vista	Plantillas Jinja2 en `src/template/` + JavaScript del cliente (`mapa.js`, `chatbot.js`).
Controlador	Funciones de ruta en `src/app.py`.

Sobre ese núcleo se añade un **subsistema RAG** (Retrieval-Augmented Generation) en el mismo proceso, con un índice vectorial en memoria persistido en archivos `.pkl`.

2. Modelo C4

2.1 Nivel 1 — Contexto del sistema

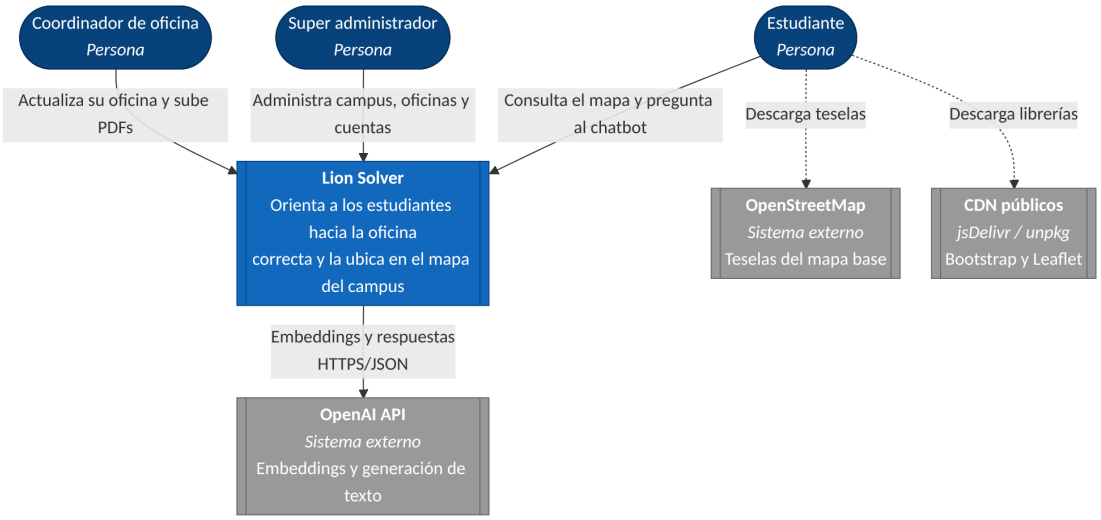


Figura 8. Nivel 1 — Contexto del sistema

2.2 Nivel 2 — Contenedores

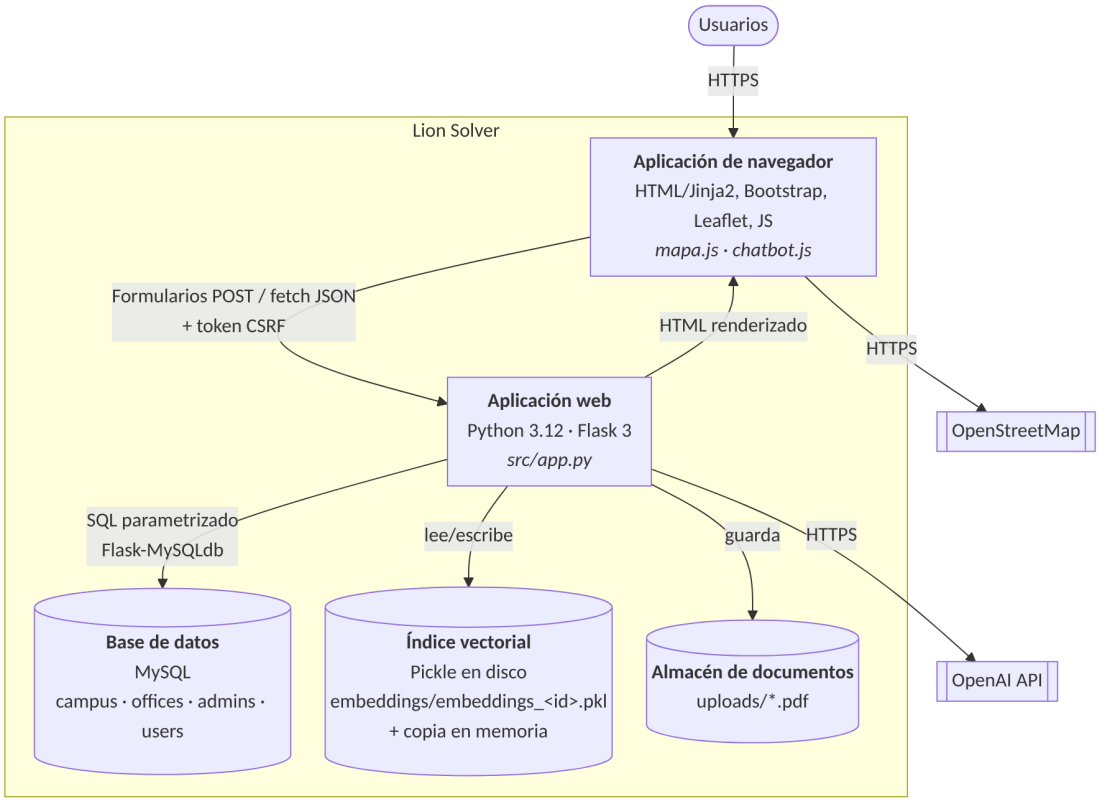


Figura 9. Nivel 2 — Contenedores

2.3 Nivel 3 — Componentes de la aplicación web

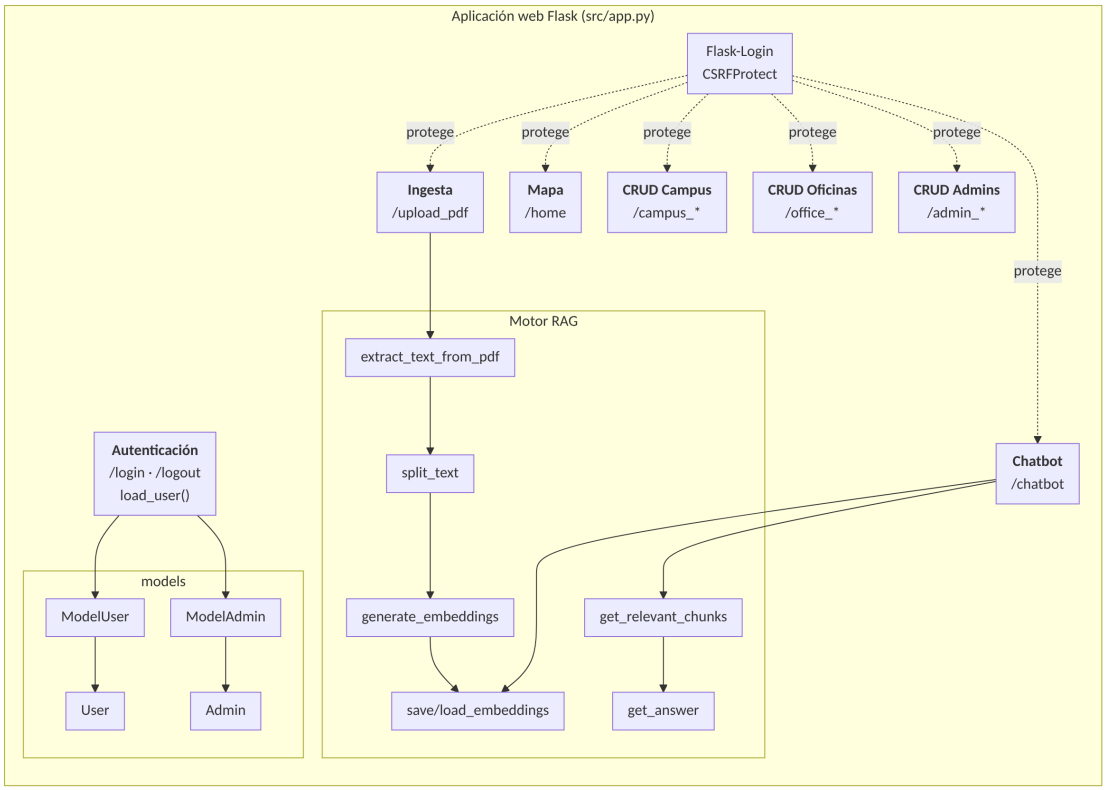


Figura 10. Nivel 3 — Componentes de la aplicación web

3. Vista de despliegue

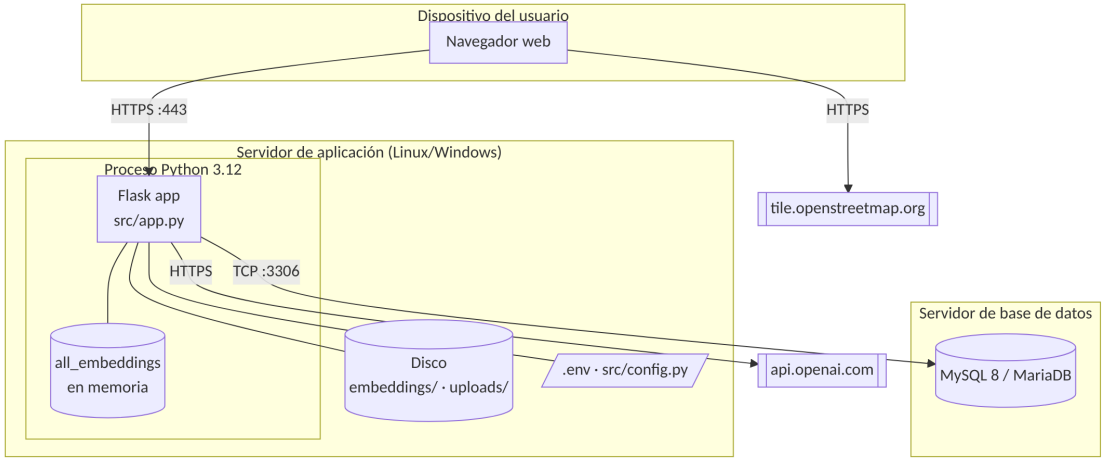


Figura 11. Vista de despliegue

En desarrollo todo corre en una sola máquina con el servidor integrado de Werkzeug. Para producción ver capítulo 9 (Instalación y despliegue).

> **Restricción importante:** el índice vectorial `all_embeddings` es una lista global del proceso. Con varios *workers* (p. ej. Gunicorn `-w 4`), un PDF subido en un worker no se refleja en los demás hasta reiniciarlos.

> Mientras no se migre a un almacén vectorial compartido, desplegar con **un solo worker** (puede usar varios *threads*).

4. Flujos principales

Flujo	Descripción breve	Diagrama
Inicio de sesión	Se busca el código en `users` y en `admins`; se valida el hash y se crea la sesión.	Secuencia 1
Consulta al chatbot	Embedding de la pregunta → top-5 por similitud coseno → prompt con contexto → respuesta + `office_id`.	Secuencia 2
Entrenamiento	PDF → texto → fragmentos ≤ 500 tokens → embeddings → `.pkl` → recarga del índice.	Secuencia 3
Visualización del mapa	Consulta campus + oficinas → JSON embebido en la plantilla → Leaflet.	Secuencia 4

5. Decisiones de arquitectura (ADR)

ADR-001 · Flask como framework web

- **Estado:** aceptada (v1.1).
- **Contexto:** equipo pequeño, necesidad de prototipar rápido con Python (mismo lenguaje que el ecosistema de IA).
- **Decisión:** Flask con extensiones (Flask-Login, Flask-WTF, Flask-MySQLdb).
- **Consecuencias:** curva de aprendizaje baja; la estructura queda a criterio del equipo, lo que derivó en un `app.py` monolítico (ver DT-04).

ADR-002 · MySQL con SQL directo (sin ORM)

- **Estado:** aceptada (v1.1).
- **Decisión:** `flask_mysqldb` con cursores y consultas parametrizadas.
- **Consecuencias:** control total del SQL; no hay migraciones versionadas ni esquema en el repositorio (se agrega anexo C (Script de base de datos)).

ADR-003 · RAG con OpenAI en lugar de *fine-tuning*

- **Estado:** aceptada (v1.1, refinada en v2.0 y v4.0).
- **Contexto:** la información cambia por oficina y con frecuencia; el personal no técnico debe poder actualizarla.
- **Decisión:** generar embeddings de fragmentos de PDF y recuperar los más similares para dar contexto a un modelo de chat.
- **Consecuencias:** actualizar conocimiento = subir un PDF; costo por consulta bajo; dependencia de un proveedor externo.

ADR-004 · Índice vectorial en archivos Pickle + memoria

- **Estado:** aceptada (v1.1), **revisar**.
- **Decisión:** un archivo `.pkl` por oficina, cargados a una lista en memoria al iniciar.
- **Consecuencias:** cero infraestructura adicional; búsqueda lineal O(n) suficiente para decenas de oficinas; no apto para múltiples procesos; `pickle` no debe cargar archivos de origen no confiable.
- **Alternativas para el futuro:** pgvector, Chroma, FAISS, SQLite-vss.

ADR-005 · Migración de Folium a Leaflet en el cliente

- **Estado:** aceptada (v3.0), sustituye la generación del mapa en servidor con Folium.
- **Contexto:** el mapa generado por Folium se insertaba como `iframe`, lo que impedía que el chatbot interactuara con él.
- **Decisión:** el servidor entrega JSON (oficinas + GeoJSON) y el cliente dibuja con Leaflet.
- **Consecuencias:** se habilita `highlightOffice()`; menor carga en servidor. `folium` sigue en `requirements.txt` sin uso (DT-03).

ADR-006 · Oficina de la respuesta = fragmento más similar (top-1)

- **Estado:** aceptada (v4.0), sustituye a "oficina más frecuente en el top-5".
- **Consecuencias:** respuesta más predecible cuando una oficina domina la pregunta; puede fallar si el mejor fragmento es genérico. Se recomienda medir con un set de evaluación (ver capítulo 8 (Chatbot RAG)).

6. Atributos de calidad y tácticas

Atributo	Táctica aplicada	Mejora propuesta
Seguridad	Hash de contraseñas, CSRF, SQL parametrizado, `@login_required`	Autorización por rol, límites de tamaño/tipo de archivo, `secure_filename`
Disponibilidad	Captura de excepciones en llamadas a OpenAI	Reintentos con *backoff*, *timeouts*
Desempeño	Índice en memoria; solo 2 llamadas a la API por pregunta	Vectorizar la similitud con NumPy; caché de embeddings de preguntas frecuentes
Mantenibilidad	Paquete `models/`	Blueprints por módulo, capa de servicios, pruebas
Portabilidad	Configuración por `.env`	Contenedor Docker

5. Diagramas UML

Todos los diagramas están en Mermaid y reflejan el código de la versión 4.1.

#	Diagrama UML	Tipo	Sección
1	Casos de uso	Comportamiento	§1
2	Clases	Estructura	§2
3	Paquetes	Estructura	§3
4	Secuencia (×5)	Interacción	§4
5	Actividad (×3)	Comportamiento	§5
6	Máquina de estados (×3)	Comportamiento	§6
7	Componentes	Estructura	§7
8	Despliegue	Estructura	§8

1. Diagrama de casos de uso

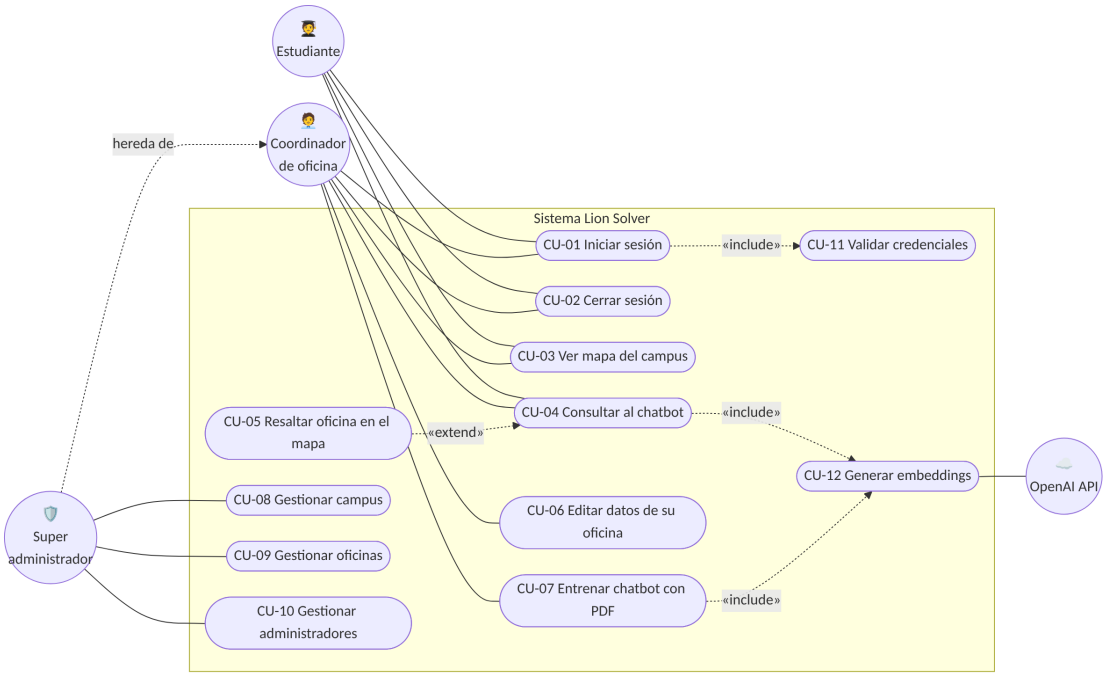


Figura 12. Diagrama de casos de uso

> Mermaid no tiene notación nativa de casos de uso; se representa con nodos ovalados (casos de uso), círculos (actores) y relaciones `«include»` / `«extend»` punteadas.

1.1 Especificación de casos de uso principales

CU-01 · Iniciar sesión

Campo	Detalle
Actor principal	Estudiante, Coordinador, Super administrador
Precondición	El usuario existe en `users` o `admins`.
Disparador	El usuario abre `/login`.
Flujo principal	1. Ingresa código y contraseña. 2. El sistema busca el código en `users` y en `admins`. 3. Valida el hash de contraseña. 4. Si es estudiante activo → `/home`; si es administrador → `/admin`.
Flujos alternos	3a. Contraseña incorrecta o estudiante inactivo → mensaje y permanece en login. 2a. Código inexistente → "Usurario no encontrado".
Postcondición	Sesión de Flask-Login creada con el `id` = código.

CU-04 · Consultar al chatbot

Campo	Detalle
Actor principal	Usuario autenticado
Actor secundario	OpenAI API
Precondición	Sesión activa; vista `/home` cargada.
Flujo principal	1. Abre la ventana del chatbot. 2. Escribe la pregunta y pulsa *Enviar* o *Enter*. 3. El cliente hace `POST /chatbot` con token CSRF. 4. El sistema obtiene el embedding de la pregunta. 5. Selecciona los 5 fragmentos más similares. 6. Envía el contexto y la pregunta al modelo de chat. 7. Devuelve `{reply, office_id}`. 8. El cliente muestra la respuesta y ejecuta CU-05.
Flujos alternos	4a. No hay embeddings cargados → mensaje "El chatbot aún no está listo…". 6a. Error del modelo → "Lo siento, ha ocurrido un error al obtener la respuesta." 3a. Error HTTP → "Lo siento, ha ocurrido un error." Pregunta = saludo exacto → respuesta fija sin llamar al modelo de chat.
Postcondición	Respuesta visible; oficina resaltada si existe en el mapa.

CU-07 · Entrenar chatbot con PDF

Campo	Detalle
Actor principal	Coordinador de oficina
Precondición	Sesión de administrador; la cuenta tiene oficina asignada.
Flujo principal	1. Abre "Entrenar Chatbot". 2. Selecciona un PDF. 3. El sistema lo guarda en `uploads/`. 4. Extrae el texto. 5. Lo divide en fragmentos ≤ 500 tokens. 6. Genera un embedding por fragmento. 7. Guarda `embeddings/embeddings_<office_id>.pkl`. 8. Recarga el índice global. 9. Muestra confirmación.
Flujos alternos	4a. Texto vacío → mensaje de error. 6a. Falla un fragmento → se omite y continúa.
Postcondición	El chatbot responde con la nueva información de la oficina.

2. Diagrama de clases

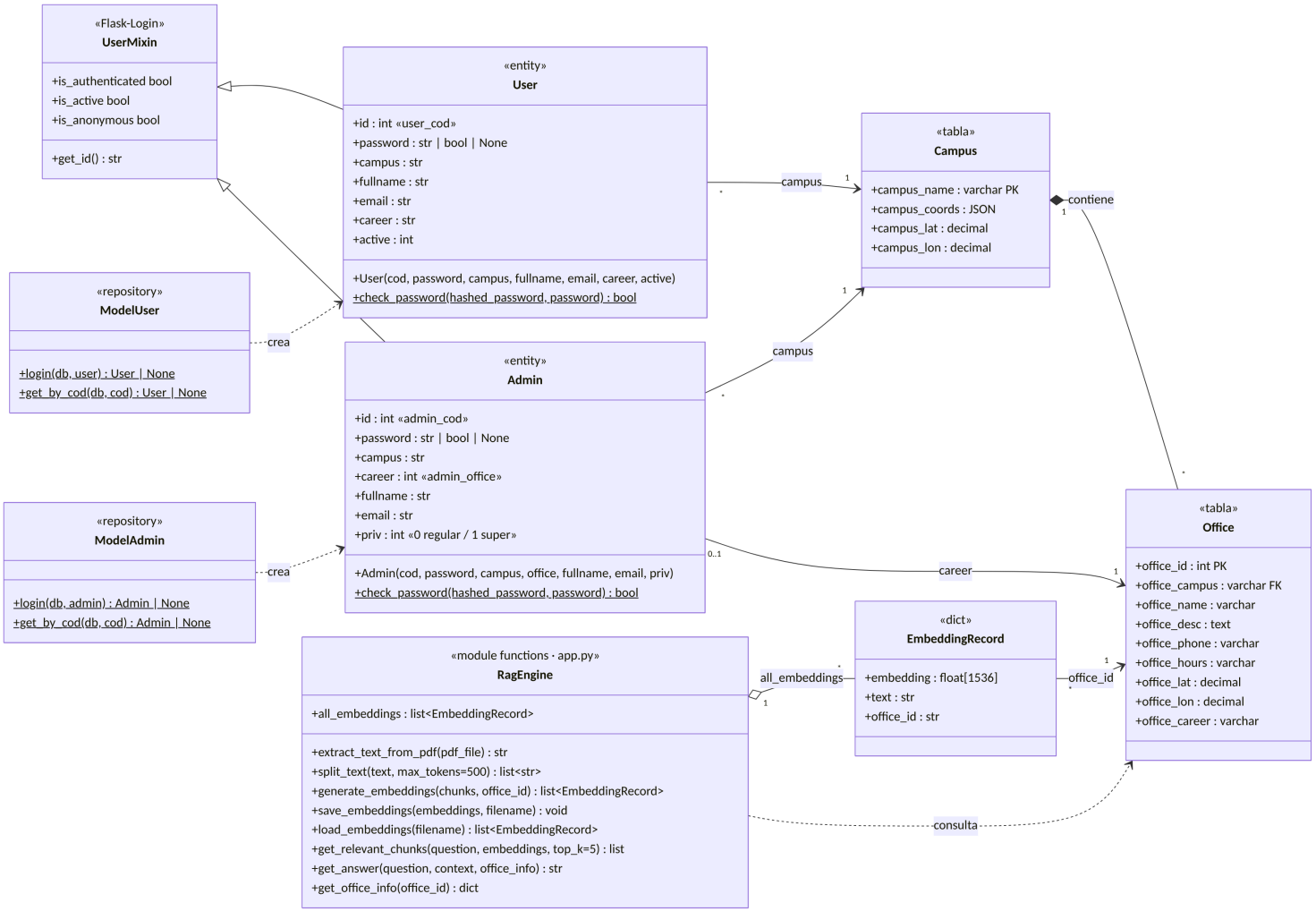


Figura 13. Diagrama de clases

Notas de diseño

- `password` en las entidades reutiliza el mismo atributo para tres significados: la contraseña en claro (al construir la credencial), el resultado booleano de `check_password` (tras el login) y `None` (al recargar desde sesión). Ver DT-05.
- En `Admin`, la oficina se guarda en el atributo `career` para que `upload_pdf` use `current_user.career` como `office_id` de los embeddings.
- `RagEngine` no es una clase real: agrupa las funciones de módulo de `src/app.py` relacionadas con el chatbot. Se modela así para mostrar sus responsabilidades.

3. Diagrama de paquetes

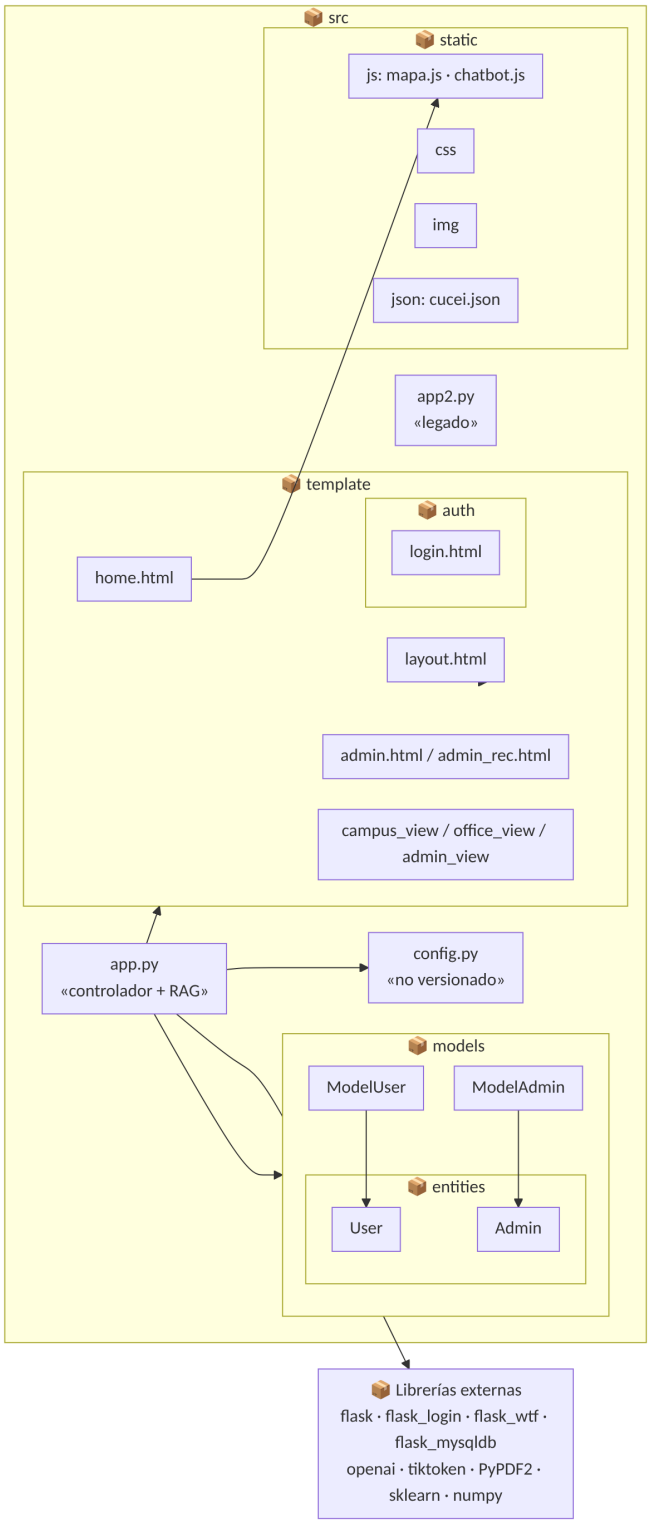


Figura 14. Diagrama de paquetes

4. Diagramas de secuencia

4.1 Inicio de sesión

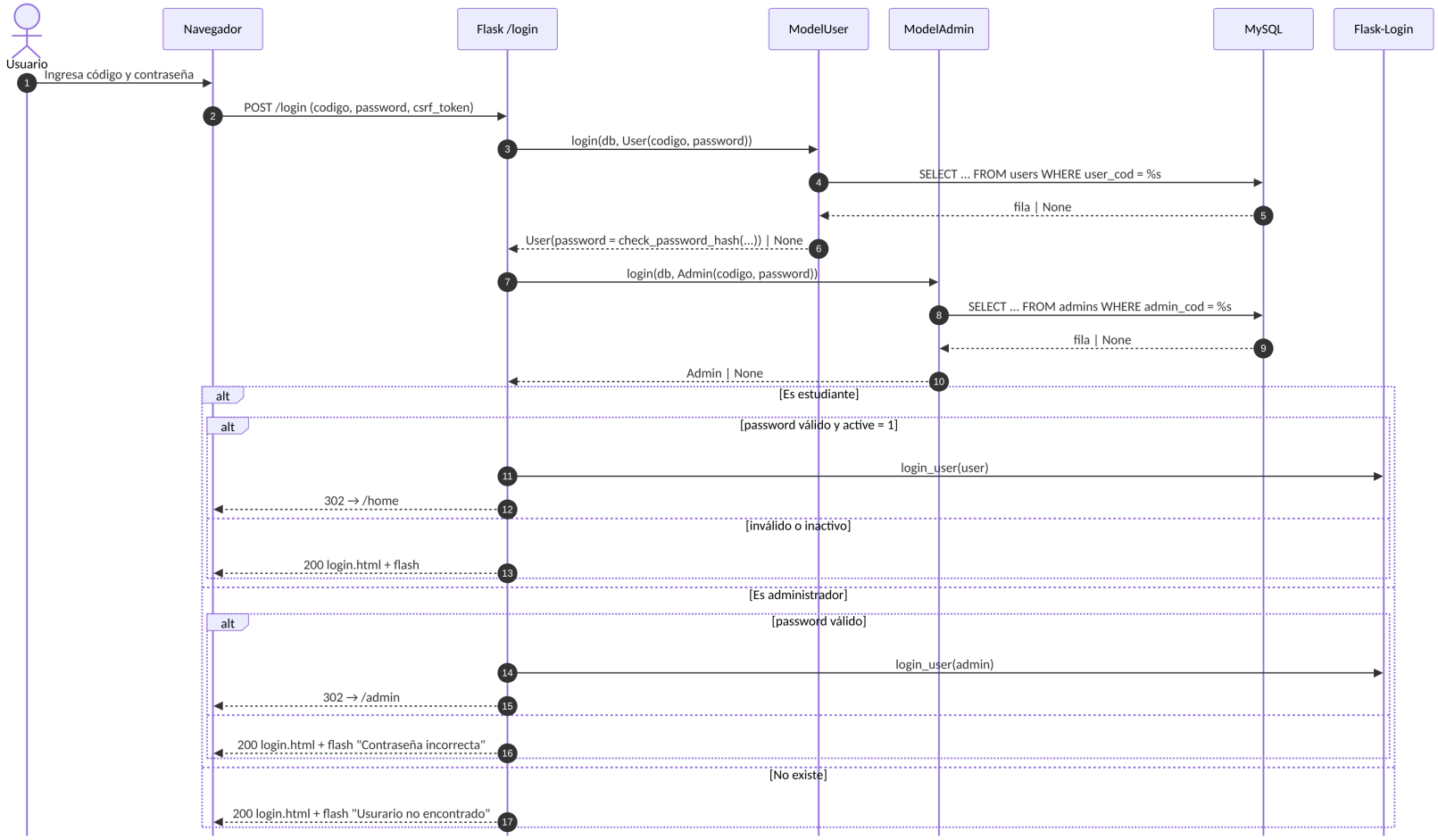


Figura 15. Inicio de sesión

4.2 Consulta al chatbot y resaltado en el mapa

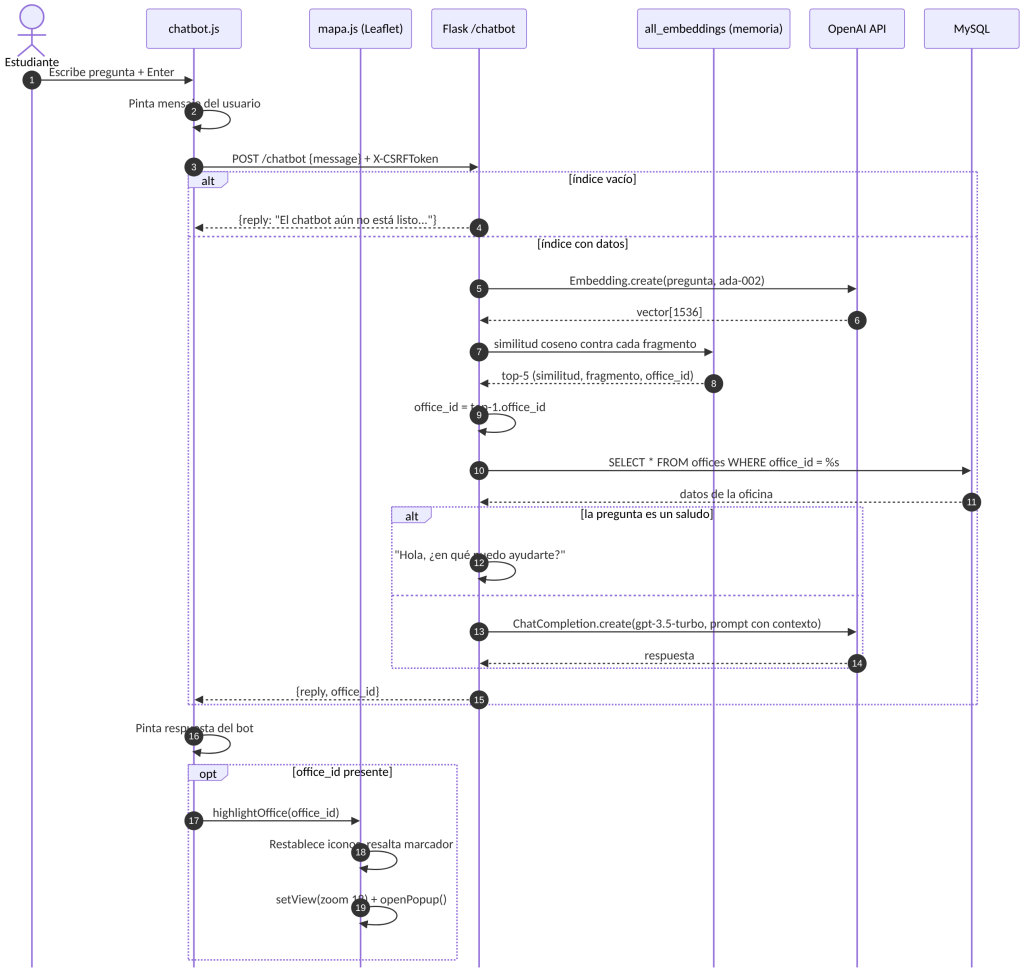


Figura 16. Consulta al chatbot y resaltado en el mapa

4.3 Entrenamiento del chatbot (carga de PDF)

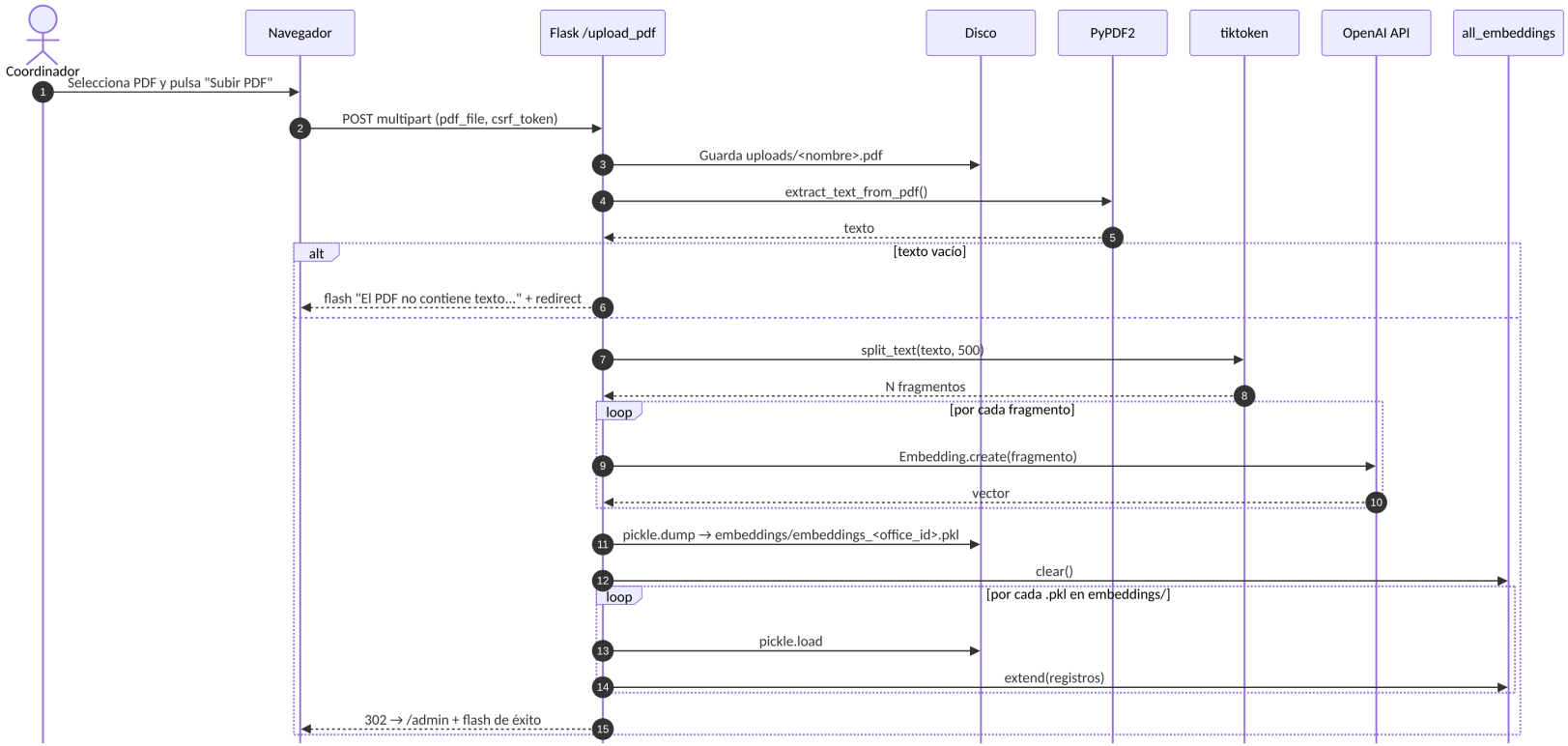


Figura 17. Entrenamiento del chatbot (carga de PDF)

4.4 Carga del mapa del campus

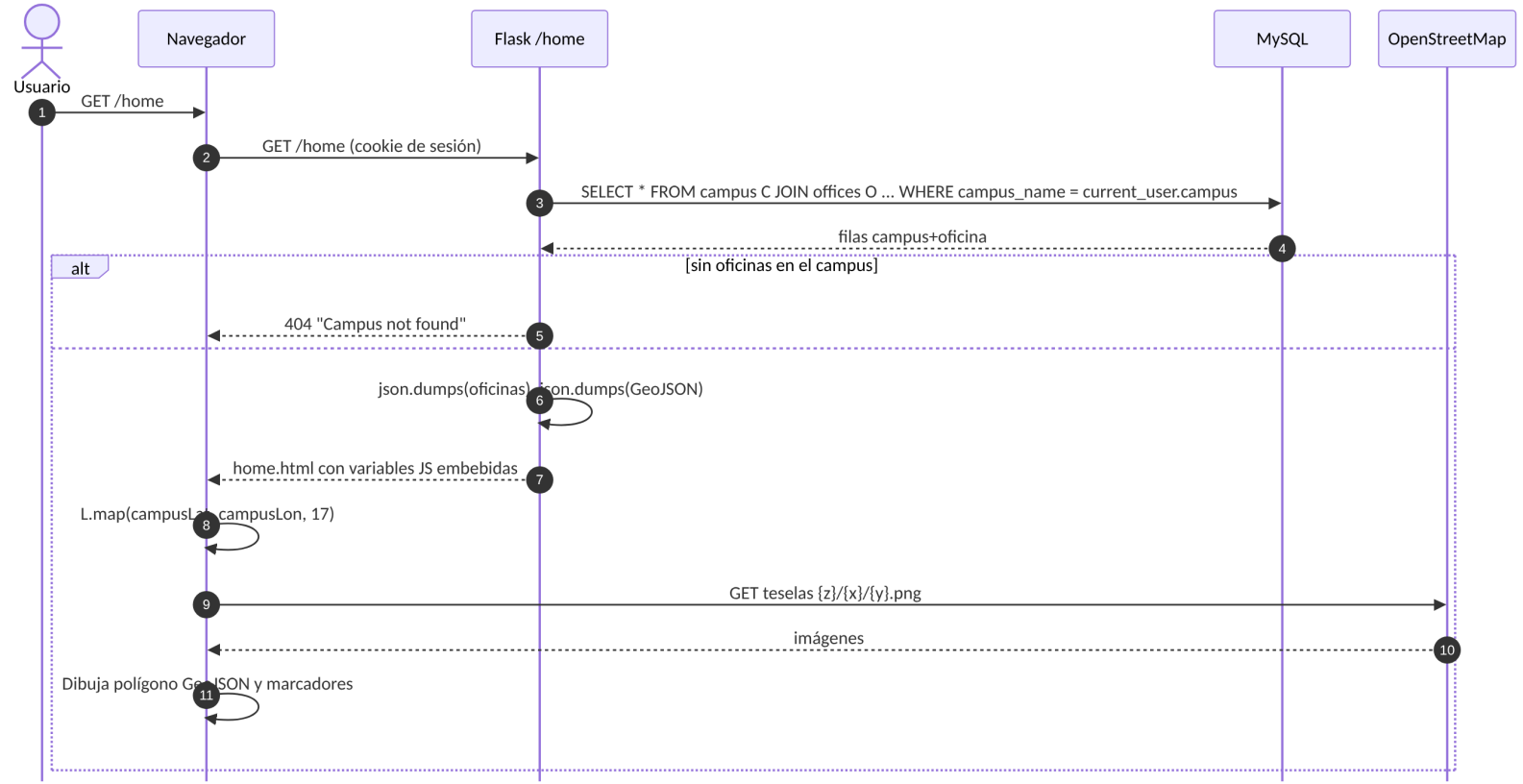


Figura 18. Carga del mapa del campus

4.5 Operación CRUD genérica (ej. alta de oficina)

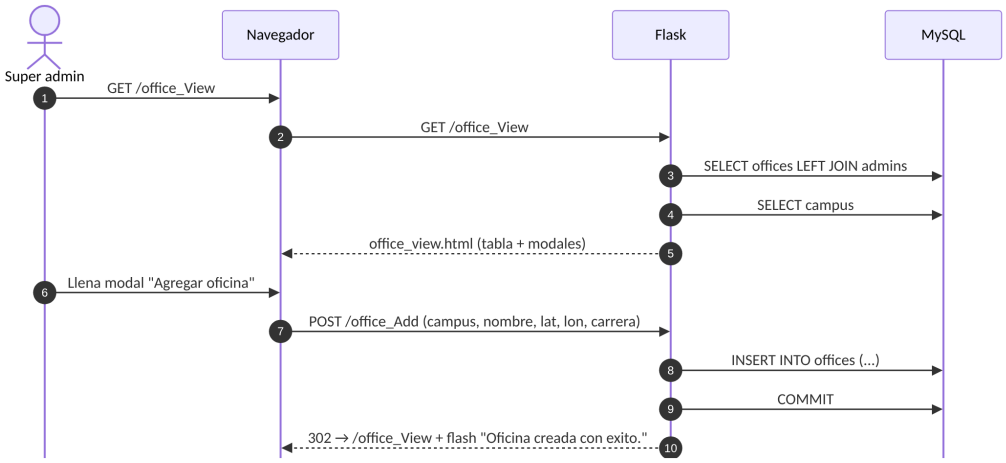


Figura 19. Operación CRUD genérica (ej. alta de oficina)

5. Diagramas de actividad

5.1 Autenticación

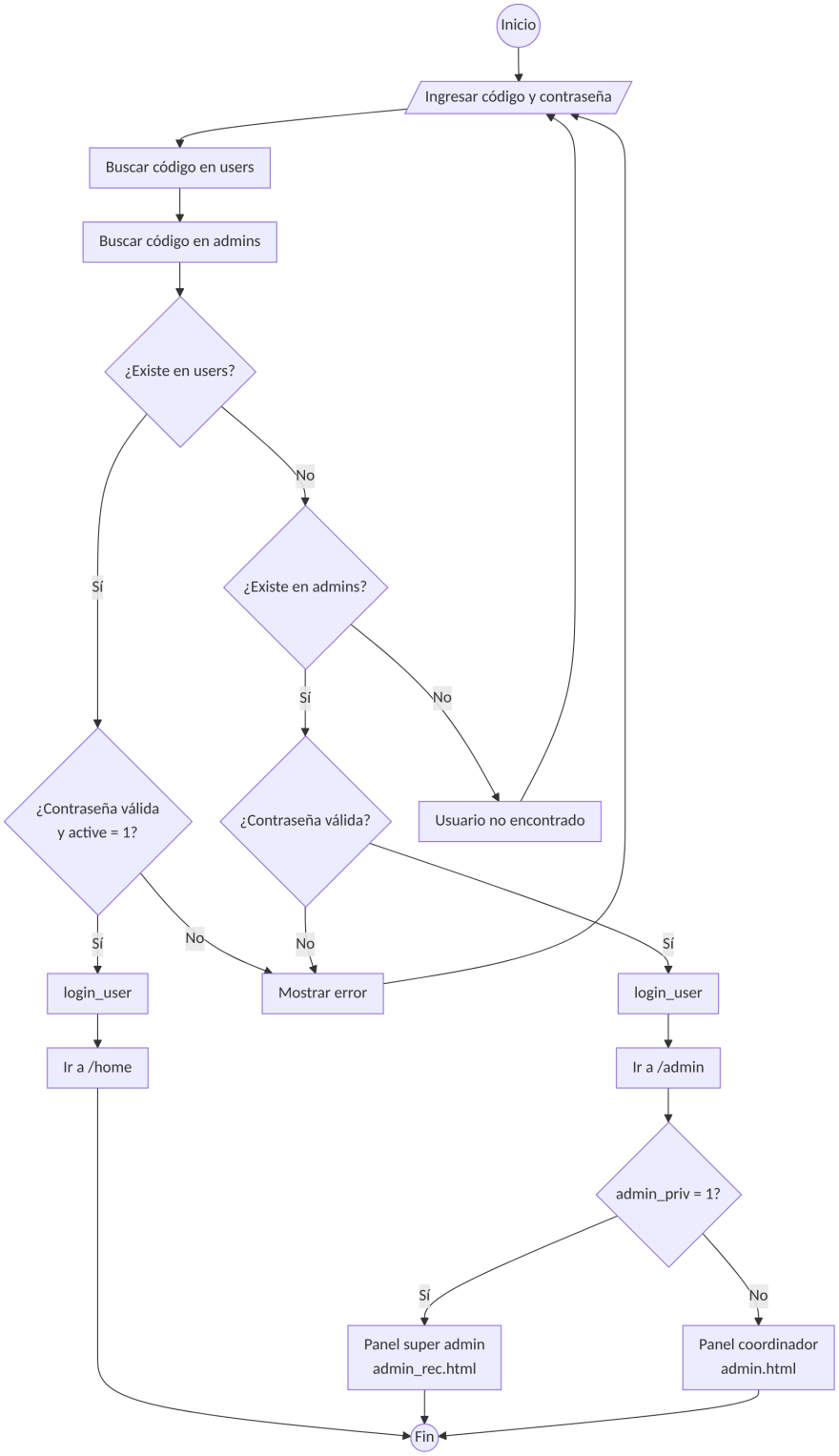


Figura 20. Autenticación

5.2 Ingesta de PDF (pipeline RAG offline)

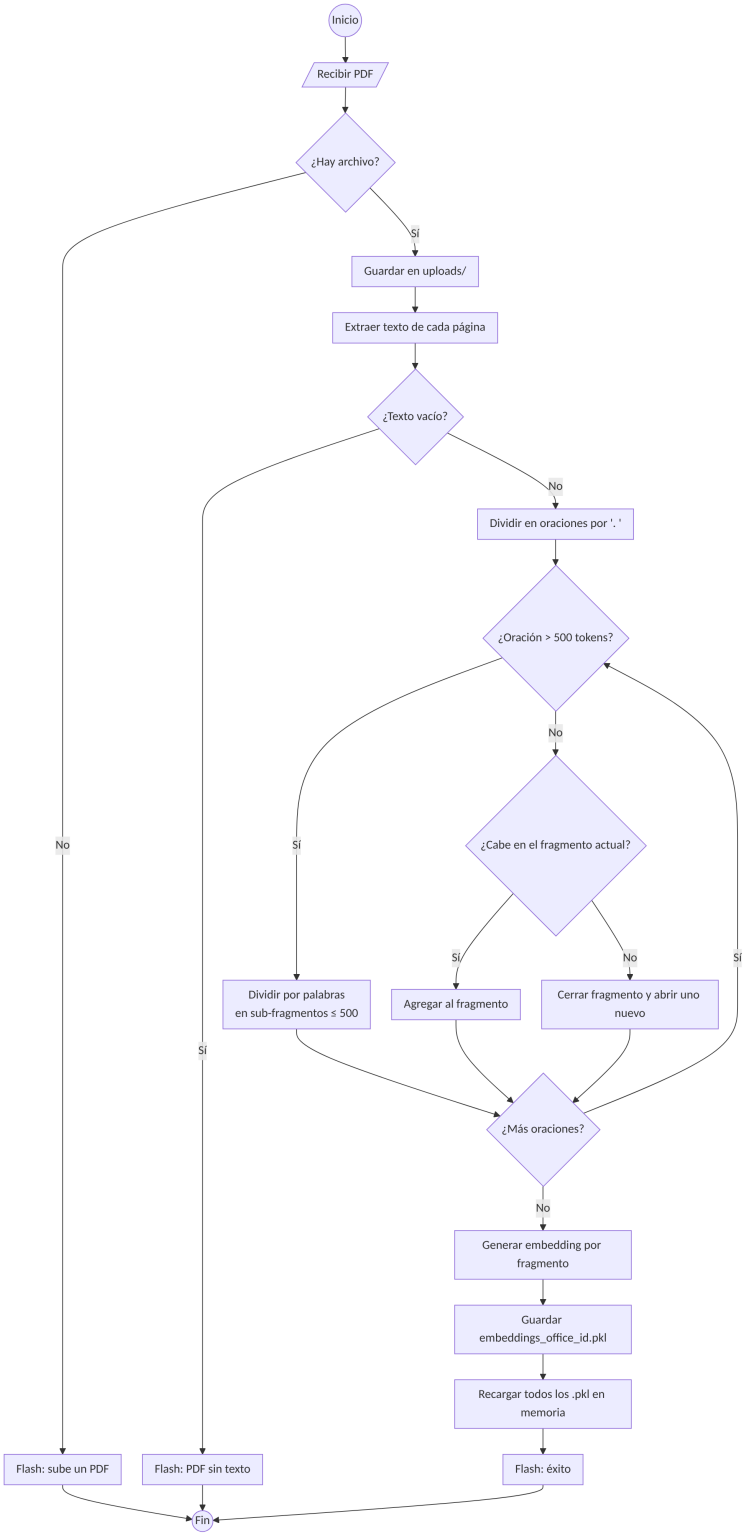


Figura 21. Ingesta de PDF (pipeline RAG offline)

5.3 Respuesta del chatbot

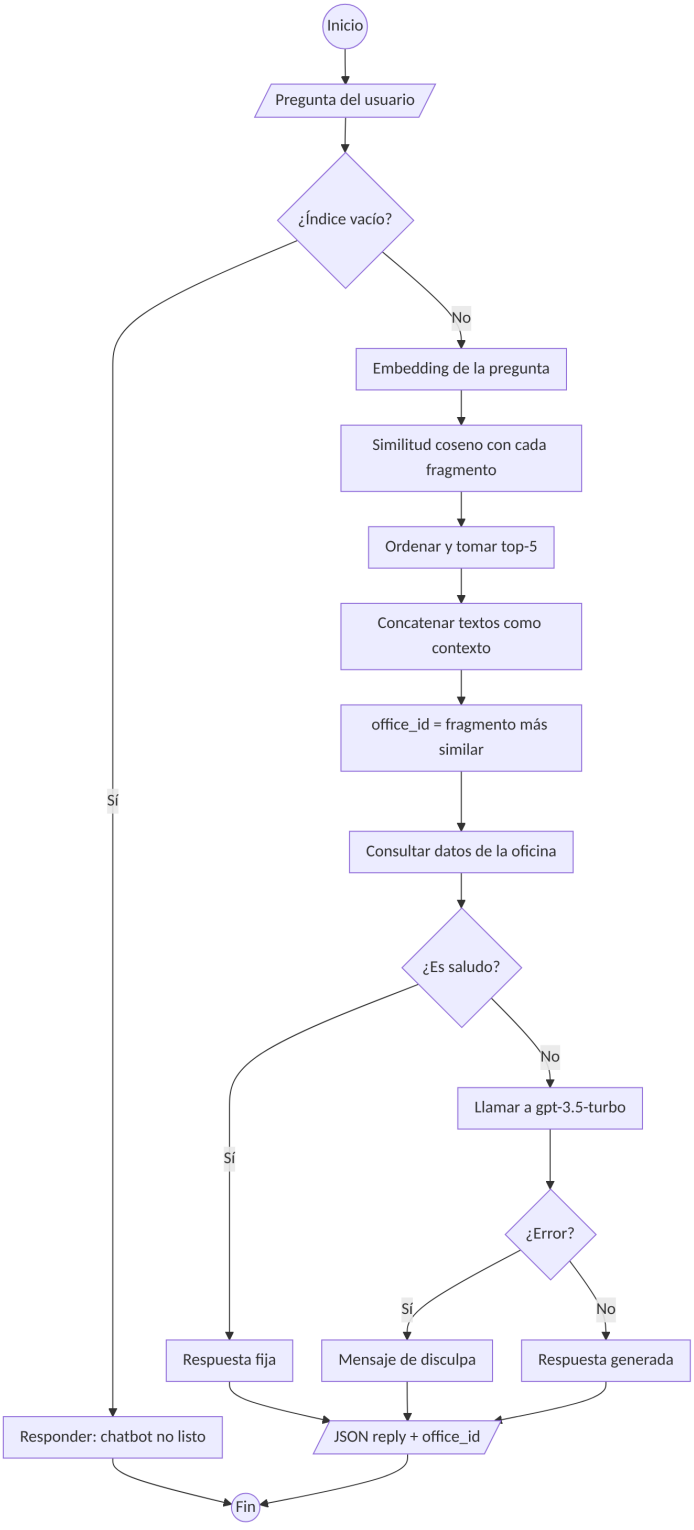


Figura 22. Respuesta del chatbot

6. Diagramas de estados

6.1 Sesión de usuario

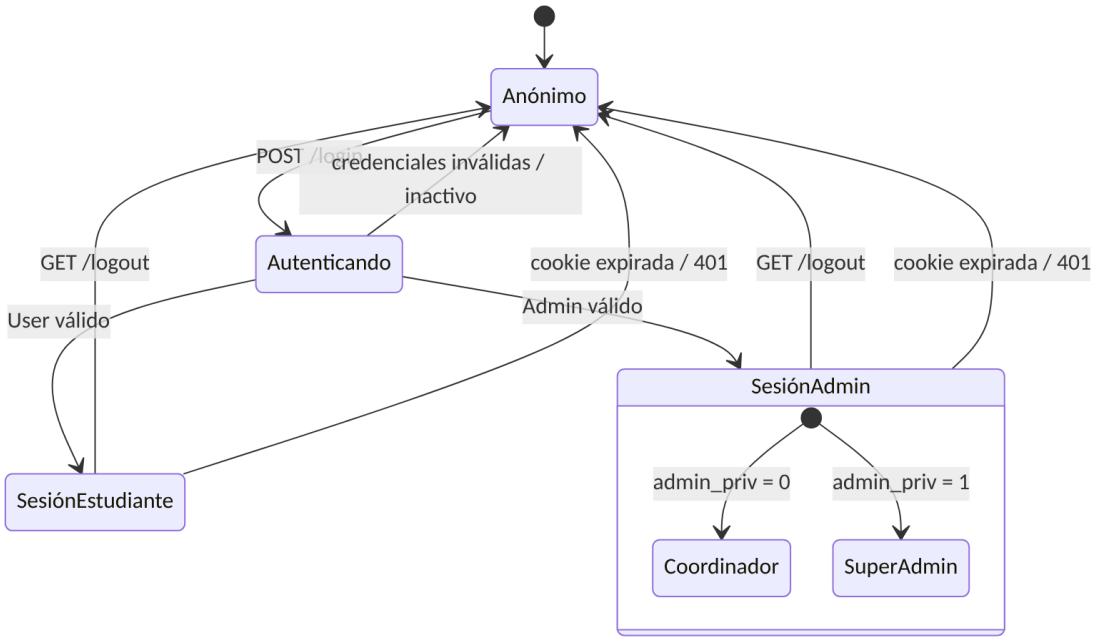


Figura 23. Sesión de usuario

6.2 Conocimiento de una oficina en el chatbot

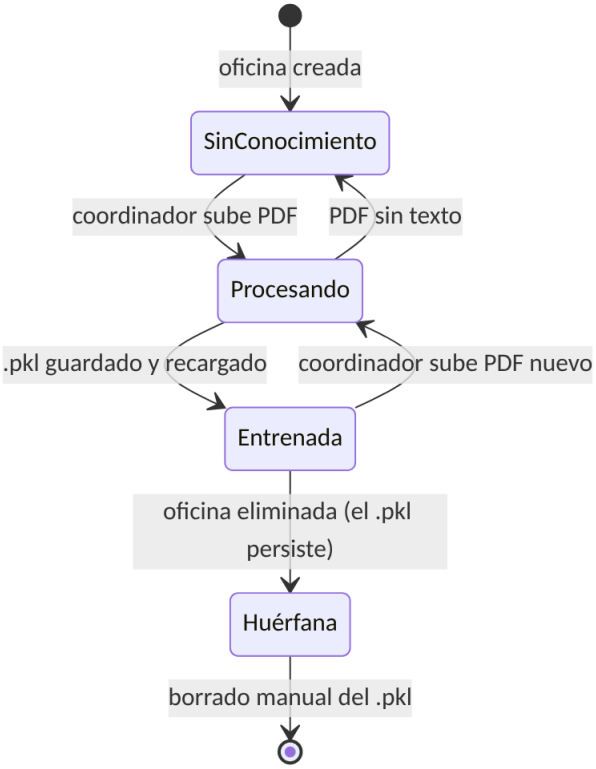


Figura 24. Conocimiento de una oficina en el chatbot

6.3 Ventana del chatbot (cliente)

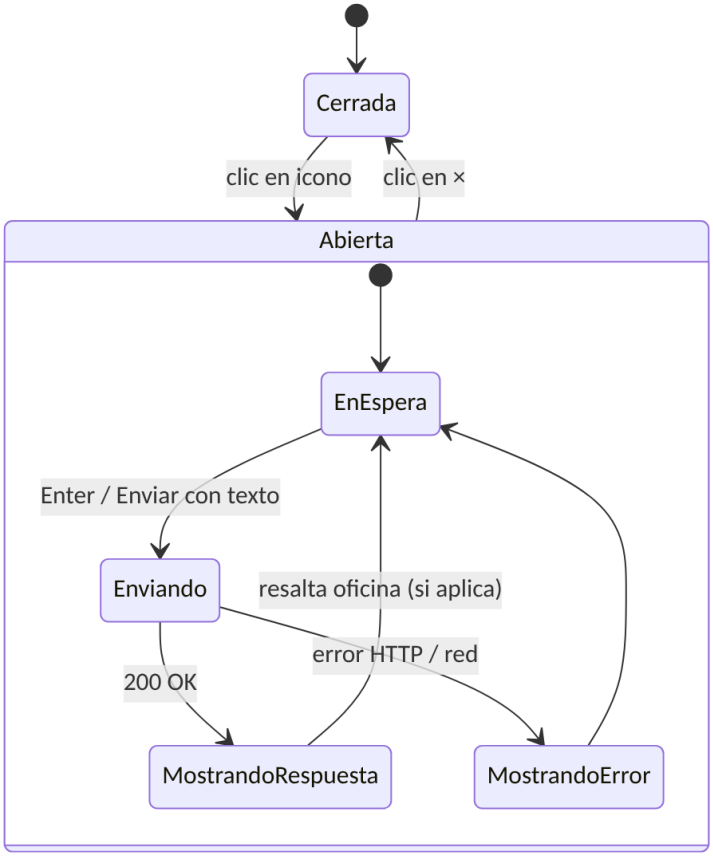


Figura 25. Ventana del chatbot (cliente)

7. Diagrama de componentes

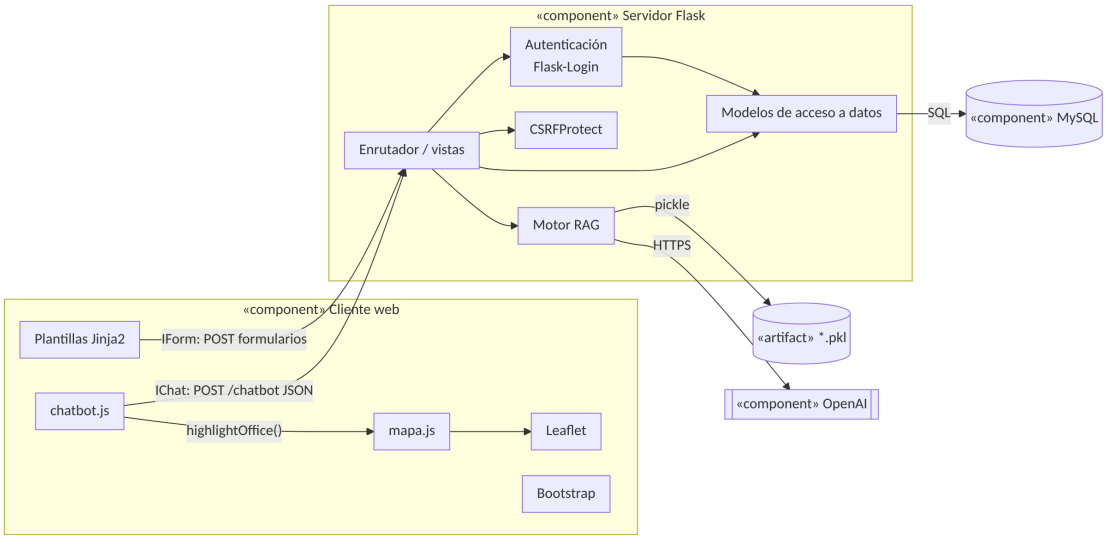


Figura 26. Diagrama de componentes

8. Diagrama de despliegue

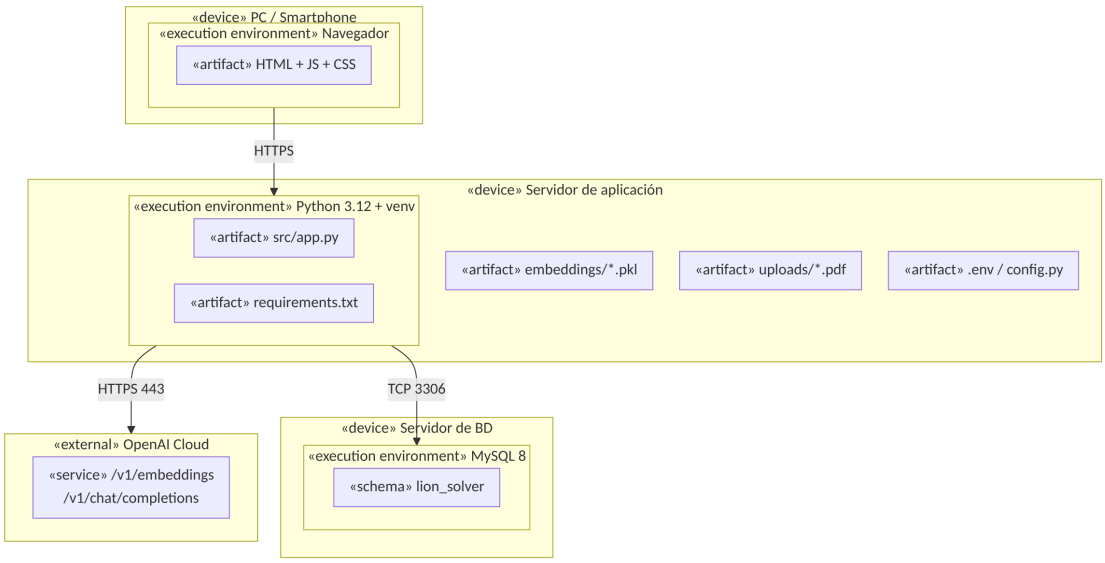


Figura 27. Diagrama de despliegue

6. Base de datos

Motor: **MySQL 8** (compatible con MariaDB 10.5+). Acceso mediante `flask_mysqldb` (`mysqlclient`) con consultas parametrizadas.

> El esquema se **reconstruyó** a partir de las consultas del código (no había DDL en el repositorio). Script listo para ejecutar: anexo C (Script de base de datos).

1. Modelo entidad-relación

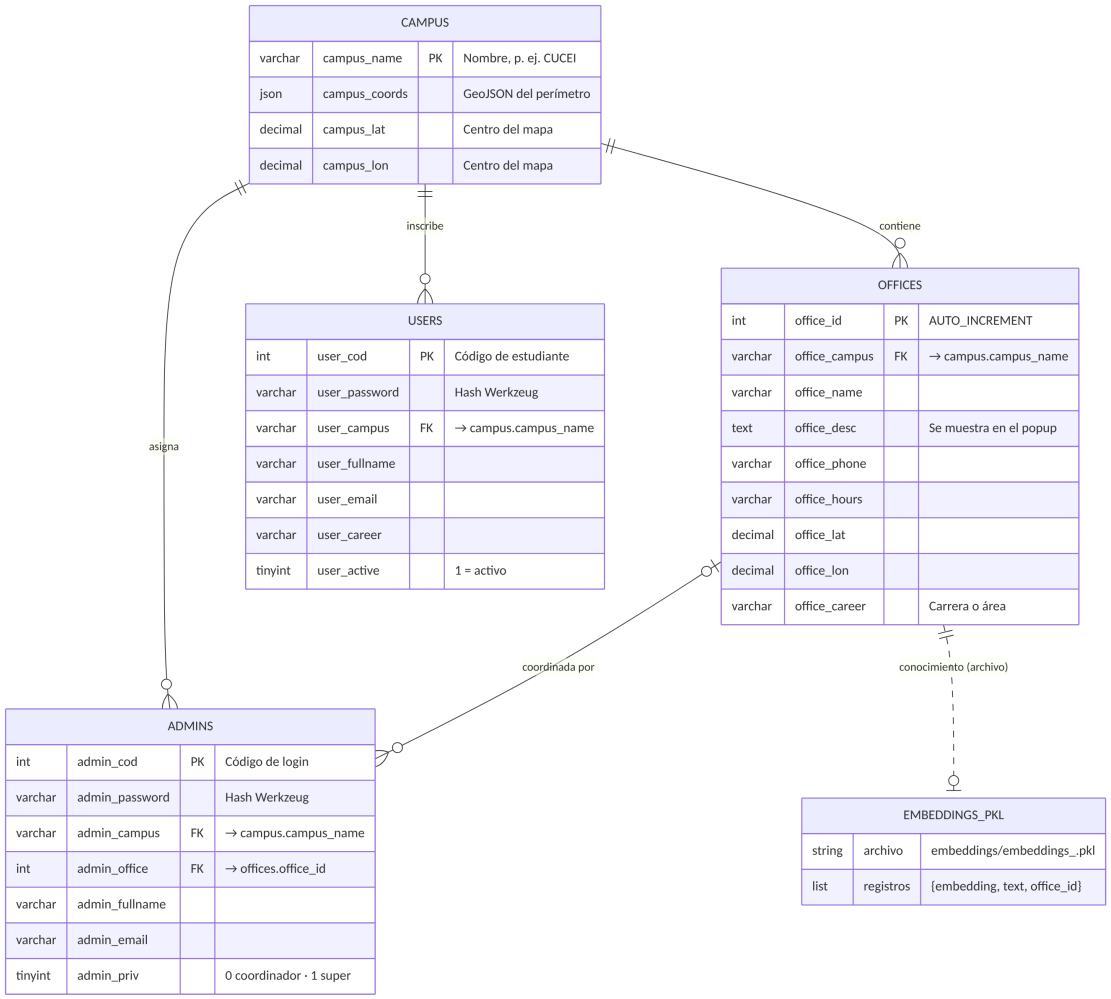


Figura 28. Modelo entidad-relación

`EMBEDDINGS_PKL` no es una tabla: representa los archivos del índice vectorial, ligados por convención de nombre a `offices.office_id`.

2. Modelo relacional

```
campus  (campus_name PK, campus_coords, campus_lat, campus_lon)
offices (office_id PK, office_campus FK→campus, office_name, office_desc,
```

```
         office_phone, office_hours, office_lat, office_lon, office_career)
admins   (admin_cod PK, admin_password, admin_campus FK→campus,
         admin_office FK→offices, admin_fullname, admin_email, admin_priv)
users    (user_cod PK, user_password, user_campus FK→campus,
         user_fullname, user_email, user_career, user_active)
```

Normalización: el esquema está en **3FN**. La clave natural de `campus` (nombre) se propaga como FK; se recomienda a futuro una clave sustituta `campus_id` para permitir renombrar campus sin cascadas.

3. Diccionario de datos

3.1 `campus`

Columna	Tipo	Nulo	Clave	Descripción	Origen en código
`campus_name`	VARCHAR(100)	No	PK	Nombre único del campus	`campus_Add`, `home`
`campus_coords`	JSON	No		`FeatureCollection` GeoJSON con el perímetro	Archivo subido en `campus_Add/campus_Edit`
`campus_lat`	DECIMAL(10,7)	No		Latitud para centrar el mapa	`mapa.js` → `setView`
`campus_lon`	DECIMAL(10,7)	No		Longitud para centrar el mapa	`mapa.js` → `setView`

3.2 `offices`

Columna	Tipo	Nulo	Clave	Descripción	Editable por
`office_id`	INT AI	No	PK	Identificador; también nombra el archivo de embeddings	—
`office_campus`	VARCHAR(100)	No	FK	Campus al que pertenece	Super admin
`office_name`	VARCHAR(150)	No		Nombre visible	Super admin, coordinador
`office_desc`	TEXT	Sí		Descripción en el popup del mapa	Coordinador
`office_phone`	VARCHAR(50)	Sí		Teléfono de contacto	Coordinador
`office_hours`	VARCHAR(100)	Sí		Horario de atención	Coordinador
`office_lat`	DECIMAL(10,7)	No		Latitud del marcador	Super admin
`office_lon`	DECIMAL(10,7)	No		Longitud del marcador	Super admin
`office_career`	VARCHAR(150)	Sí		Carrera/área que atiende	Super admin

3.3 `admins`

Columna	Tipo	Nulo	Clave	Descripción
`admin_cod`	INT	No	PK	Código de inicio de sesión
`admin_password`	VARCHAR(255)	No		Hash generado con `generate_password_hash`
`admin_campus`	VARCHAR(100)	No	FK	Campus del administrador (define el mapa que ve)
`admin_office`	INT	Sí	FK	Oficina que coordina; se usa como `office_id` al subir PDF

Columna	Tipo	Nulo	Clave	Descripción
admin_fullname	VARCHAR(150)	No		Nombre completo
admin_email	VARCHAR(150)	No		Correo institucional
admin_priv	TINYINT(1)	No		1 super admin · 0 coordinador

3.4 users

Columna	Tipo	Nulo	Clave	Descripción
user_cod	INT	No	PK	Código de estudiante
user_password	VARCHAR(255)	No		Hash Werkzeug
user_campus	VARCHAR(100)	No	FK	Campus cuyo mapa verá
user_fullname	VARCHAR(150)	No		Nombre completo (se muestra en el encabezado del mapa)
user_email	VARCHAR(150)	No		Correo
user_career	VARCHAR(150)	Sí		Carrera
user_active	TINYINT(1)	No		1 permite iniciar sesión

4. Consultas principales

Uso	Consulta	Ubicación
Login estudiante	`SELECT user_cod, user_password, … FROM users WHERE user_cod = %s`	src/models/ModelUser.py:8
Login admin	`SELECT admin_cod, admin_password, … FROM admins WHERE admin_cod = %s`	src/models/ModelAdmin.py:8
Mapa	`SELECT * FROM campus C INNER JOIN offices O ON C.campus_name = O.office_campus WHERE campus_name = %s`	src/app.py:268
Panel admin	`SELECT * FROM admins A INNER JOIN offices O … INNER JOIN campus C … WHERE admin_cod = %s`	src/app.py:554
Listado de oficinas	`SELECT * FROM offices O LEFT JOIN admins A ON O.office_id = A.admin_office`	src/app.py:399
Info para chatbot	`SELECT * FROM offices WHERE office_id = %s`	src/app.py:374

5. Consideraciones de integridad

Situación	Comportamiento con el DDL propuesto	Recomendación
Borrar un campus con oficinas	Las oficinas se eliminan en cascada	Pedir confirmación explícita en la UI
Borrar un campus con admins o estudiantes	Se bloquea (RESTRICT) y la app muestra error 500	Capturar IntegrityError y mostrar un mensaje
Borrar una oficina con coordinador	admin_office queda en NULL	El panel /admin usa INNER JOIN y fallaría: reasignar antes
Borrar una oficina con embeddings	El .pkl sigue en disco y el chatbot puede seguir sugiriéndola	Eliminar embeddings_<id>.pkl al borrar la oficina

Situación	Comportamiento con el DDL propuesto	Recomendación
Mismo código en `users` y `admins`	Permitido por el esquema	Unificar en una tabla de cuentas o validar unicidad cruzada (SEC-05)

6. Respaldo

```
# Respaldo
mysqldump -u <usuario> -p --single-transaction --routines lion_solver > lion_solver_$
(date +%F).sql
# Restauración
mysql -u <usuario> -p lion_solver < lion_solver_YYYY-MM-DD.sql
```

Respalde también las carpetas `embeddings/` y `uploads/`, que forman parte del estado del sistema.

7. Referencia de rutas HTTP

Todas las rutas están definidas en `src/app.py`. Las respuestas son HTML renderizado salvo `/chatbot`, que responde JSON. Todos los `POST` requieren el campo `csrf_token` (formularios) o la cabecera `X-CSRFToken` (fetch).

Leyenda de acceso: [Público] público · [Sesión] cualquier sesión (`@login_required`) · [Est] pensado para estudiante · [Coord] pensado para coordinador · [Super] pensado para super admin.

> ⚠️ En la versión 4.1 **no se verifica el rol** en el servidor: cualquier sesión puede llamar las rutas [Super] y [Coord] si conoce la URL. Ver SEC-01 en capítulo 12 (Seguridad, riesgos y deuda técnica).

1. Resumen

Método	Ruta	Acceso	Respuesta	Descripción
GET	`/`	[Público]	302 → `/home`	Redirección a inicio
GET	`/login`	[Público]	`auth/login.html`	Formulario de acceso
POST	`/login`	[Público]	302 / `auth/login.html`	Autenticación
GET	`/logout`	[Público]	302 → `/login`	Cierra sesión
GET	`/home`	[Sesión] [Est]	`home.html` / 404	Mapa + chatbot
POST	`/chatbot`	[Sesión] [Est]	JSON	Pregunta al chatbot
GET, POST	`/admin`	[Sesión] [Coord]	`admin.html` o `admin_rec.html`	Panel según privilegio
POST	`/office_Edit`	[Sesión] [Coord]	302 → `/admin`	Edita datos de contacto de la oficina
POST	`/upload_pdf`	[Sesión] [Coord]	302 → `/admin`	Entrena el chatbot con un PDF
GET	`/campus_view`	[Sesión] [Super]	`campus_view.html`	Lista de campus
POST	`/campus_Add`	[Sesión] [Super]	302 → `/admin`	Alta de campus
POST	`/campus_Edit`	[Sesión] [Super]	302 → `/admin`	Edición de campus
POST	`/campus_Delete`	[Sesión] [Super]	302 → `/admin`	Baja de campus
GET, POST	`/office_View`	[Sesión] [Super]	`office_view.html`	Lista de oficinas
POST	`/office_Add`	[Sesión] [Super]	302 → `/office_View`	Alta de oficina
POST	`/office_Edit_Rec`	[Sesión] [Super]	302 → `/office_View`	Edición de ubicación/carrera
POST	`/office_Delete`	[Sesión] [Super]	302 → `/office_View`	Baja de oficina
GET, POST	`/admin_View`	[Sesión] [Super]	`admin_view.html`	Lista de administradores
POST	`/admin_Add`	[Sesión] [Super]	302 → `/admin_View`	Alta de administrador
POST	`/admin_Edit`	[Sesión] [Super]	302 → `/admin_View`	Edición de administrador
POST	`/admin_Delete`	[Sesión] [Super]	302 → `/admin_View`	Baja de administrador
GET	`/protected`	[Sesión]	HTML	Vista de prueba

2. Detalle

`POST /login`

Campo (form)	Tipo	Requerido	Descripción
`codigo`	número	Sí	Código de estudiante o administrador
`password`	texto	Sí	Contraseña
`csrf_token`	texto	Sí	Token CSRF

Resultado	Respuesta
Estudiante activo	`302 /home`
Administrador	`302 /admin`
Contraseña incorrecta / inactivo	`200` login + “Contraseña incorrecta o Estudiante no activo”
No existe	`200` login + “Usurario no encontrado”

`GET /home`

Consulta el campus de `current_user.campus` y sus oficinas. Inyecta en la plantilla:

Variable JS	Contenido
`offices`	Arreglo de filas `campus ⋈ offices`
`campusLat`, `campusLon`	Centro del mapa
`campusGeoJSON`	Polígono del campus
`csrfToken`	Token para `/chatbot`

Devuelve `404 "Campus not found"` si el campus no tiene oficinas.

`POST /chatbot`

Petición

```
POST /chatbot HTTP/1.1
Content-Type: application/json
X-CSRFToken: <token>

{ "message": "¿Dónde entrego el reporte de servicio social?" }
```

Respuesta 200

```
{
  "reply": "Debes entregar tu reporte en la Unidad de Servicio Social ... Puedes
contactarla al ...",
  "office_id": "6"
}
```

Caso	Respuesta
Sin embeddings cargados	`{"reply": "El chatbot aún no está listo. Por favor, procesa primero el PDF."}`
Error en OpenAI (chat)	`{"reply": "Lo siento, ha ocurrido un error al obtener la respuesta.", "office_id": "…"}`

Caso	Respuesta
Error en OpenAI (embedding de la pregunta)	`500` (excepción no capturada)
Sin sesión	`302 /login`

`POST /upload_pdf`

`multipart/form-data` con `pdf_file` (PDF) y `csrf_token`. El `office_id` se toma de la oficina del administrador en sesión (`current_user.career`). Genera `embeddings/embeddings_<office_id>.pkl`, recarga el índice y redirige a `/admin`.

> `GET /upload_pdf` intenta renderizar `upload_pdf.html`, plantilla que no existe en el repositorio (la carga se hace desde un modal en `admin.html`). Ver DT-07.

`POST /office_Edit`

Campo	Descripción
`office_id`	Oficina del coordinador (oculto)
`office_name`, `office_desc`, `office_phone`, `office_hours`	Nuevos valores (todos se sobrescriben)

`POST /campus_Add` • `POST /campus_Edit`

`multipart/form-data`:

Campo	Add	Edit	Descripción
`campus_name`	✔️	✔️ (oculto)	Clave del campus
`campus_lat`, `campus_lon`	✔️	✔️	Centro del mapa
`campus_coords`	✔️ archivo	✔️ archivo	GeoJSON del perímetro (`.json`)

`POST /campus_Delete`

`campus_name`.

`POST /office_Add` • `POST /office_Edit_Rec` • `POST /office_Delete`

Campo	Add	Edit_Rec	Delete
`office_id`		✔️	✔️
`office_campus`	✔️		
`office_name`	✔️	✔️	
`office_lat`, `office_lon`	✔️	✔️	
`office_career`	✔️	✔️	

`POST /admin_Add` • `POST /admin_Edit` • `POST /admin_Delete`

Campo	Add	Edit	Delete
`admin_cod`	✔️	✔️ (clave)	✔️
`admin_password`	✔️ (se guarda con hash)		

Campo	Add	Edit	Delete
`admin_fullname`, `admin_email`	✔️	✔️	
`admin_campus`	✔️	✔️	
`admin_office`	✔️	✔️	
`admin_priv`	✔️ (`1/0`)	✔️	

3. Manejo de errores

Código	Manejador	Resultado
401	`status_401`	Redirección a `/login`
404	`status_404`	`<h1>Pagina no encontrada</h1>`
400 (CSRF inválido)	Flask-WTF	Página de error por defecto

Los manejadores y `CSRFProtect` se registran dentro de `if __name__ == '__main__'`, por lo que **solo están activos al ejecutar `python src/app.py`**.

4. Mapa de navegación

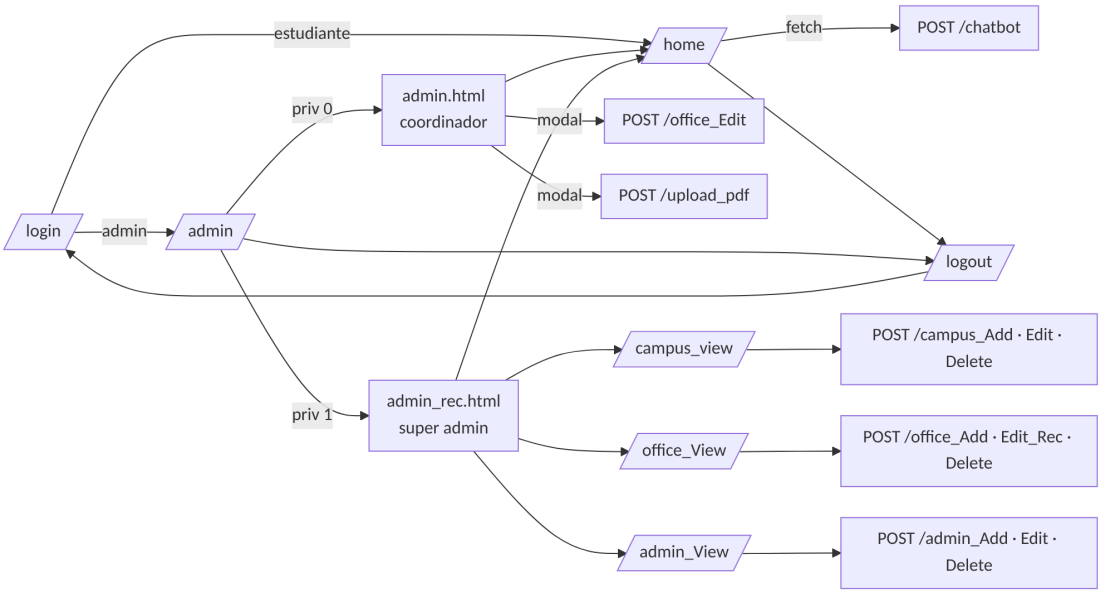


Figura 29. Mapa de navegación

8. Chatbot RAG

El chatbot de Lion Solver usa **Retrieval-Augmented Generation (RAG)**: en lugar de reentrenar un modelo, recupera los fragmentos más relevantes de los documentos oficiales de cada oficina y se los pasa como contexto a un modelo de lenguaje.

1. Visión general

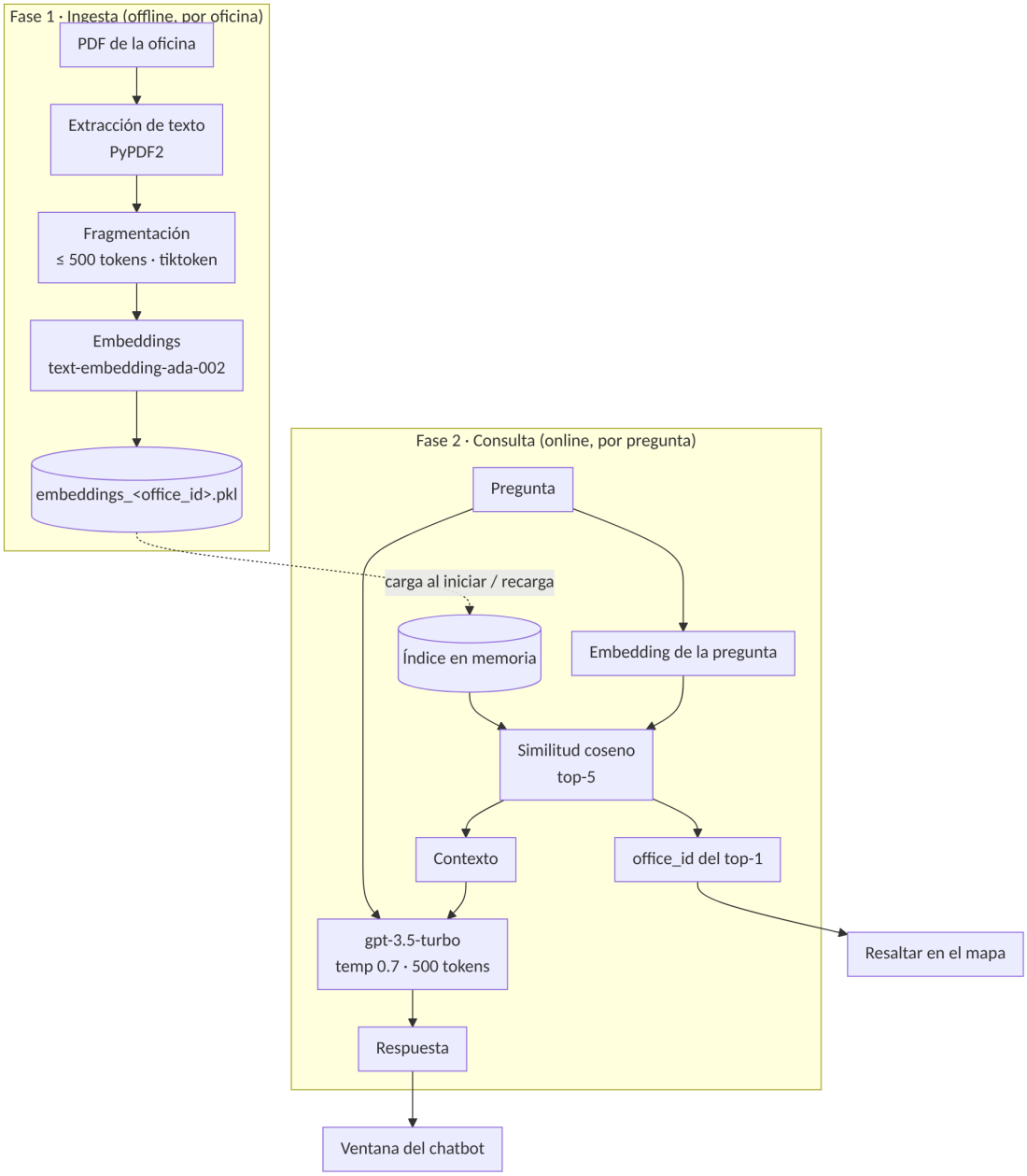


Figura 30. Visión general del flujo RAG

2. Parámetros

Parámetro	Valor	Ubicación
Modelo de embeddings	`text-embedding-ada-002` (1 536 dimensiones)	`src/app.py:96`, `:127`
Modelo de chat	`gpt-3.5-turbo`	`src/app.py:166`
Tokenizador	`cl100k_base`	`src/app.py:51`
Tamaño máximo de fragmento	500 tokens	`split_text(max_tokens=500)`
Fragmentos recuperados (`top_k`)	5	`src/app.py:575`
`temperature`	0.7	`src/app.py:171`
`max_tokens` de la respuesta	500	`src/app.py:170`
SDK	`openai==0.28.0` (API heredada)	`requirements.txt`
Variable de entorno	`OPENAI_API_KEY`	`.env`

3. Fase de ingesta

1. **Recepción** — `POST /upload_pdf` guarda el archivo en `uploads/<nombre>`.
2. **Extracción** — `extract_text_from_pdf()` recorre las páginas con `PyPDF2.PdfReader(strict=False)` y concatena el texto. Los PDF escaneados (solo imagen) producen texto vacío y se rechazan.
3. **Fragmentación** — `split_text()`:
   - divide el texto por `". "` (oraciones);
   - acumula oraciones mientras la suma de tokens sea ≤ 500;
   - si una sola oración excede 500 tokens, la parte por palabras en sub-fragmentos.
4. **Vectorización** — `generate_embeddings()` llama a la API una vez por fragmento. Si un fragmento falla, se registra y se omite.
5. **Persistencia** — `save_embeddings()` serializa con `pickle` una lista de registros:

```
{"embedding": [0.0123, -0.0456, ...],  # 1536 floats
 "text": "Fragmento del documento...",
 "office_id": "6"}                      # str(current_user.career)
```

6. **Recarga** — se vacía `all_embeddings` y se vuelven a cargar todos los `.pkl` de `embeddings/`.

Un archivo por oficina ⇒ subir un PDF nuevo **reemplaza** el conocimiento anterior de esa oficina. Para combinar varios documentos, únalos en un solo PDF antes de subirlo.

4. Fase de consulta

1. El cliente envía `{"message": "..."}` a `POST /chatbot`.
2. Si `all_embeddings` está vacío, responde que el chatbot no está listo.
3. `get_relevant_chunks()` obtiene el embedding de la pregunta y calcula `cosine_similarity` contra cada registro (búsqueda lineal), ordena de mayor a menor y devuelve los 5 primeros.
4. Se concatena el texto de los 5 fragmentos como **contexto**.
5. `office_id` = oficina del fragmento **más similar** (top-1).
6. `get_office_info()` consulta la oficina en MySQL.

7. `get_answer()`:
   - si la pregunta es exactamente un saludo (`hola`, `buenos días`, `hi`, …) devuelve *"Hola, ¿en qué puedo ayudarte?"*;
   - en otro caso envía el *prompt* a `gpt-3.5-turbo`.
8. Responde `{"reply": ..., "office_id": ...}`; el cliente llama a `highlightOffice()`.

4.1 Prompt actual

```
Utiliza el siguiente contexto para responder la pregunta de manera clara y concisa.
Al final de tu respuesta, indica cuál es la oficina correspondiente y cómo el estudiante
puede contactarla.

Contexto:
{context}

Pregunta:
{question}

Respuesta:
```

> Observación: `office_info` (nombre, horario y teléfono reales de la oficina) se recibe en `get_answer()` pero **no se incluye en el prompt**, por lo que el modelo solo puede indicar datos de contacto si aparecen en el PDF. Ver HU-16.

5. Costos y desempeño

Operación	Llamadas a la API	Comentario
Subir un PDF de *N* fragmentos	*N* embeddings	Un PDF de 20 páginas ≈ 20–40 fragmentos
Una pregunta	1 embedding + 1 chat completion	El saludo exacto ahorra la llamada de chat pero **no** la de embedding

La búsqueda es O(n) en Python puro por cada fragmento. Para cientos de fragmentos es despreciable frente a la latencia de la API; para miles conviene vectorizar con NumPy (`matrix @ q / norms`) o usar un índice ANN (FAISS, pgvector).

6. Evaluación recomendada

Para medir la calidad de forma objetiva antes de cambiar parámetros:

1. Crear un conjunto de 30–50 preguntas reales con la oficina esperada.
2. Medir **Precisión@1 de oficina** (¿`office_id` correcto?) y **Recall@5 de fragmento**.
3. Revisar manualmente una muestra de respuestas: exactitud, si inventa datos (alucinación), si menciona contacto.
4. Repetir tras cada cambio de `top_k`, tamaño de fragmento, modelo o prompt.

Métrica	Meta propuesta
Precisión@1 de oficina	≥ 0.80
Respuestas sin datos inventados	≥ 0.95
Latencia p95	< 8 s

7. Mejoras propuestas

Prioridad	Mejora	Beneficio
Alta	Migrar a `openai>=1.x` y a `text-embedding-3-small` + un modelo de chat vigente	Continuidad del servicio, menor costo
Alta	Inyectar `office_info` en el prompt	Respuestas con contacto y horario correctos
Alta	Instrucción “si el contexto no contiene la respuesta, dilo” y `temperature` ≤ 0.3	Menos alucinaciones
Media	Umbral mínimo de similitud para devolver `office_id`	Evita resaltar oficinas irrelevantes
Media	Fragmentos con solapamiento (p. ej. 50 tokens)	Mejor recuperación en bordes de fragmento
Media	Almacén vectorial compartido (pgvector/Chroma)	Varios workers, sin `pickle`
Baja	Historial de conversación por sesión	Preguntas de seguimiento
Baja	*Streaming* de la respuesta	Percepción de velocidad

9. Instalación y despliegue

1. Requisitos

Componente	Versión	Notas
Python	3.12	Las dependencias fijadas en `requirements.txt` requieren ≥ 3.9
MySQL / MariaDB	8.0 / 10.5+	Soporte para columnas `JSON`
Cabeceras de cliente MySQL	—	Necesarias para compilar `mysqlclient`
Cuenta de OpenAI	—	Clave de API con saldo
Git	2.x	

Dependencias del sistema para `mysqlclient`:

```
# Debian / Ubuntu
sudo apt-get install python3-dev default-libmysqlclient-dev build-essential pkg-config
# macOS (Homebrew)
brew install mysql-client pkg-config
# Windows: pip instala ruedas precompiladas de mysqlclient
```

2. Instalación local

```
# 1. Clonar
git clone https://github.com/KevinPonce90/LionSolver.git
cd LionSolver

# 2. Entorno virtual
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate

# 3. Dependencias
pip install -r requirements.txt
```

> `requirements.txt` está codificado en **UTF-16**. `pip` lo lee sin problema; si una herramienta falla al leerlo, conviértalo con `iconv -f UTF-16 -t UTF-8 requirements.txt > requirements-utf8.txt`.

3. Configuración

3.1 Variables de entorno (`.env` en la raíz)

```
cp .env.example .env
```

Variable	Obligatoria	Descripción
`OPENAI_API_KEY`	Sí	Clave de la API de OpenAI (leída por `src/app.py`)
`SECRET_KEY`	Sí	Clave de sesión/CSRF (leída por `src/config.example.py`)
`MYSQL_HOST`, `MYSQL_PORT`, `MYSQL_USER`, `MYSQL_PASSWORD`, `MYSQL_DB`	Sí	Conexión a MySQL (leídas por `src/config.example.py`)

Si ya tiene un `src/config.py` propio con los valores escritos directamente, solo `OPENAI_API_KEY` es necesaria en `.env`.

3.2 Configuración de Flask (`src/config.py`)

`src/app.py` importa `from config import config` y usa `config['development']`. El archivo **no se versiona** (está en `.gitignore`); parta de la plantilla:

```
cp src/config.example.py src/config.py
```

La plantilla lee los valores de `.env` y define dos perfiles, `development` y `production`.

Clave	Descripción
`SECRET_KEY`	Firma de la cookie de sesión y de los tokens CSRF. Use un valor aleatorio largo.
`DEBUG`	`True` solo en desarrollo.
`MYSQL_HOST`, `MYSQL_PORT`	Servidor de base de datos.
`MYSQL_USER`, `MYSQL_PASSWORD`	Credenciales.
`MYSQL_DB`	Nombre de la base (por defecto `lion_solver`).

Genere una `SECRET_KEY`:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

3.3 Base de datos

```
mysql -u root -p < docs/database/schema.sql
```

Cree un usuario de aplicación con privilegios mínimos:

```
CREATE USER 'lion_app'@'localhost' IDENTIFIED BY '<contraseña-segura>';
GRANT SELECT, INSERT, UPDATE, DELETE ON lion_solver.* TO 'lion_app'@'localhost';
FLUSH PRIVILEGES;
```

Cargue el primer campus y el primer super administrador (ver comentarios al final de `schema.sql`). Para el GeoJSON de CUCEI use `src/static/json/cucei.json`.

Para generar el hash de una contraseña:

```
python -c "from werkzeug.security import generate_password_hash as g;
print(g('CambiaEsto123'))"
```

4. Ejecución

Desde la **raíz del repositorio**:

```
python src/app.py
```

Abra http://127.0.0.1:5000.

> **No use `flask run`.** La configuración, `CSRFProtect` y los manejadores de errores se registran dentro de `if __name__ == '__main__':` en `src/app.py`; con `flask run` no se cargarían.

Al iniciar verá en consola cuántos embeddings se cargaron desde `embeddings/`.

5. Despliegue en producción

5.1 Lista de verificación previa

- ☐ `DEBUG = False` y `SECRET_KEY` aleatoria.
- ☐ Control de acceso por rol implementado (SEC-01).
- ☐ HTTPS obligatorio (terminado en el *reverse proxy*).
- ☐ `SESSION_COOKIE_SECURE = True`, `SESSION_COOKIE_HTTPONLY = True`, `SESSION_COOKIE_SAMESITE = "Lax"`.
- ☐ `MAX_CONTENT_LENGTH` configurado (p. ej. 10 MB) para limitar el tamaño de PDFs.
- ☐ Usuario de base de datos con privilegios mínimos.
- ☐ Respaldos programados de MySQL, `embeddings/` y `uploads/`.
- ☐ Monitoreo de consumo y límite de gasto en la cuenta de OpenAI.

5.2 Servidor WSGI

El índice vectorial vive en la memoria del proceso; use **un solo worker** con varios hilos (ver capítulo 4 (Arquitectura del sistema)). Como la configuración se aplica en el bloque `__main__`, cree un punto de entrada WSGI que la replique, por ejemplo `src/wsgi.py`:

```
from app import app, csrf, status_401, status_404
from config import config

app.config.from_object(config['production'])
csrf.init_app(app)
app.register_error_handler(401, status_401)
app.register_error_handler(404, status_404)
pip install gunicorn
cd /ruta/LionSolver
gunicorn --chdir src --workers 1 --threads 8 --bind 127.0.0.1:8000 wsgi:app
```

> Con `--chdir src` el directorio de trabajo cambia a `src/`; las rutas relativas `embeddings/` y `uploads/` se resolverían ahí. Mueva esas carpetas o, mejor, conviértalas en rutas absolutas configurables (DT-09).

5.3 Nginx como *reverse proxy*

```
server {
    listen 443 ssl http2;
    server_name lionsolver.ejemplo.edu.mx;

    ssl_certificate     /etc/letsencrypt/live/lionsolver/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/lionsolver/privkey.pem;

    client_max_body_size 10m;

    location /static/ {
        alias /ruta/LionSolver/src/static/;
        expires 7d;
    }

    location / {
        proxy_pass         http://127.0.0.1:8000;
        proxy_set_header   Host $host;
        proxy_set_header   X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }
}
```

5.4 Servicio systemd

```
[Unit]
Description=Lion Solver
After=network.target mysql.service

[Service]
User=lionsolver
WorkingDirectory=/ruta/LionSolver
EnvironmentFile=/ruta/LionSolver/.env
ExecStart=/ruta/LionSolver/.venv/bin/gunicorn --chdir src --workers 1 --threads 8 --bind
127.0.0.1:8000 wsgi:app
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

6. Solución de problemas

Síntoma	Causa probable	Solución
`ModuleNotFoundError: No module named 'config'`	Falta `src/config.py`	Copie `src/config.example.py`
`ModuleNotFoundError: No module named 'models'`	Se ejecuta con otro directorio en `sys.path`	Ejecute `python src/app.py` desde la raíz
`OperationalError (2002/1045)`	MySQL apagado o credenciales incorrectas	Revise `MYSQL_*` en `config.py`
Error al instalar `mysqlclient`	Faltan cabeceras de MySQL	Instale las dependencias del sistema (§1)
"El chatbot aún no está listo"	`embeddings/` vacío	Suba un PDF desde el panel de un coordinador
`openai.error.AuthenticationError`	`OPENAI_API_KEY` ausente o inválida	Revise `.env`
"Campus not found" (404) en `/home`	El campus del usuario no tiene oficinas	Registre al menos una oficina
`400 Bad Request: The CSRF token is missing`	Formulario sin `csrf_token`	Incluya `{{ csrf_token() }}` en el formulario
`TemplateNotFound: upload_pdf.html`	Se accedió a `GET /upload_pdf` o el PDF no tenía texto	Use el modal del panel; ver DT-07

10. Manual de usuario

1. Acceso al sistema

1. Abra la dirección de Lion Solver en su navegador (computadora o celular).
2. Escriba su **código** (número de estudiante o de administrador) y su **contraseña**.
3. Pulse **Iniciar sesión**.

Mensaje	Significado	Qué hacer
“Usurario no encontrado”	El código no está registrado	Verifique el código o solicite el alta al administrador
“Contraseña incorrecta o Estudiante no activo”	Contraseña errónea o cuenta desactivada	Reintente o contacte al administrador
“Contraseña incorrecta”	(administradores) contraseña errónea	Reintente

Para salir, pulse **Cerrar sesión** en la esquina superior derecha.

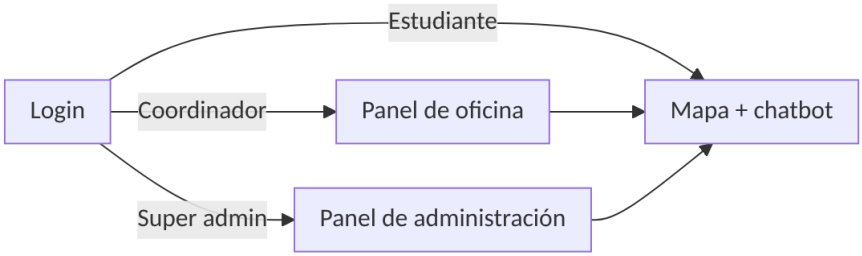


Figura 31. Navegación según el rol

2. Estudiantes

2.1 Mapa del campus

Al entrar verá el nombre de su campus y un mapa con:

- **Contorno del campus** — línea negra con relleno azul claro.
- **Marcadores de oficinas** — pulse uno para ver su nombre y descripción.
- **Zoom** — use los botones +/–, la rueda del ratón o el gesto de pellizco.

2.2 Preguntar al chatbot

1. Pulse el **icono del león** en la esquina inferior.
2. Escriba su pregunta, por ejemplo:
   - *“¿Qué necesito para liberar mi servicio social?”*
   - *“¿Dónde entrego mi reporte bimestral?”*
3. Pulse **Enviar** o la tecla **Enter**.
4. Lea la respuesta. Si el sistema identifica una oficina, el mapa **se acerca a ella**, cambia su marcador por uno **resaltado** y abre su información.
5. Cierre la ventana con **×**; el mapa sigue visible.

Consejos para mejores respuestas

- Haga preguntas concretas y completas (una duda por mensaje).
- Mencione el trámite por su nombre (servicio social, titulación, constancia…).
- El chatbot no recuerda mensajes anteriores: incluya el contexto en cada pregunta.
- Si la respuesta no es útil, reformule la pregunta o acuda a la oficina resaltada.

3. Coordinadores de oficina

Al iniciar sesión verá tres tarjetas:

Tarjeta	Acción
Ver Mapa	Abre el mapa del campus con el chatbot (igual que un estudiante).
Editar oficina	Abre un formulario para actualizar nombre, descripción, teléfono y horario.
Entrenar Chatbot	Permite subir el PDF con la información de su oficina.

3.1 Editar la oficina

1. Pulse **Editar Oficina**.
2. Complete **todos** los campos: nombre, descripción, teléfono y horario.
3. Pulse **Guardar cambios**.

> ⚠️ El formulario se abre vacío y guarda todos los campos; si deja uno en blanco, se borrará el valor actual. Escriba de nuevo los datos que no quiera cambiar.

La **descripción** es el texto que verán los estudiantes al pulsar el marcador.

3.2 Entrenar el chatbot

1. Prepare **un solo PDF** con toda la información de su oficina: trámites, requisitos, fechas, ubicación, teléfono, correo y horario.
2. Pulse **Entrenar Chatbot**, seleccione el archivo y pulse **Subir PDF**.
3. Espere el mensaje *"El PDF ha sido procesado y los embeddings se han generado correctamente."* (puede tardar de segundos a un par de minutos según el tamaño).

Buenas prácticas para el PDF

Hacer	Evitar
PDF generado desde Word/Docs (texto seleccionable)	PDF escaneado o fotos de documentos
Frases completas terminadas en punto	Tablas complejas o texto en columnas
Incluir el nombre de la oficina y sus datos de contacto	Información de otras oficinas
Un documento que reúna todo	Subir varios PDF por separado (cada carga **reemplaza** la anterior)

4. Super administradores

El panel muestra cuatro tarjetas: **Ver Mapa**, **Ver Campus**, **Ver Oficinas** y **Ver Administradores**. Cada listado tiene un botón para **agregar** y, por fila, botones para **editar** y **eliminar**.

4.1 Campus

Campo	Descripción
Nombre	Identificador único (no se puede cambiar después).
Latitud / Longitud	Punto donde se centrará el mapa. Se obtiene, por ejemplo, con clic derecho en Google Maps u OpenStreetMap.
Archivo de coordenadas	Archivo `.json` en formato GeoJSON con el contorno del campus. Puede dibujarse en https://geojson.io y exportarse. Ejemplo: `src/static/json/cucei.json`.

> Eliminar un campus elimina sus oficinas; no podrá eliminarse si tiene estudiantes o administradores asignados.

4.2 Oficinas

Campo	Descripción
Campus	Campus al que pertenece.
Nombre	Nombre visible en el mapa.
Latitud / Longitud	Ubicación exacta del marcador (debe estar dentro del contorno del campus).
Carrera	Carrera o área que atiende.

La descripción, el teléfono y el horario los captura el **coordinador** de la oficina. La columna **Coordinador** muestra quién la administra.

4.3 Administradores

Campo	Descripción
Código	Usuario de inicio de sesión. No debe coincidir con un código de estudiante.
Contraseña	Solo al crear; se guarda cifrada.
Nombre y correo	Datos de contacto.
Campus	Define el mapa que verá.
Oficina	Oficina que coordina y a la que se asocian sus PDFs.
Privilegio	**Super** (administración completa) o **Regular** (coordinador).

Flujo recomendado para dar de alta una oficina nueva:

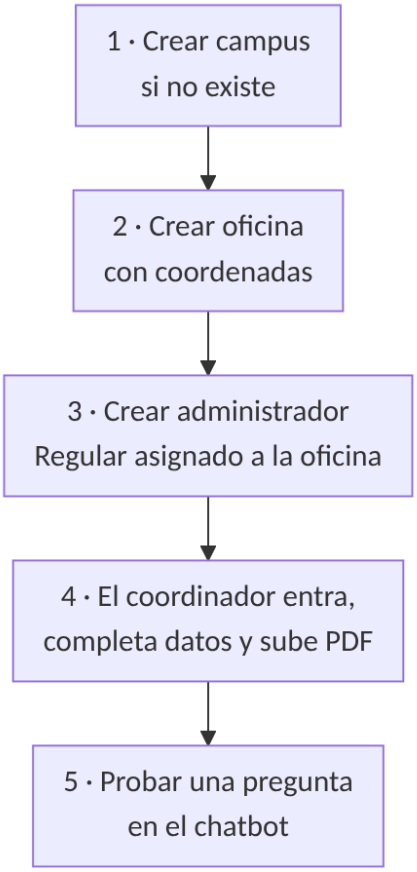


Figura 32. Flujo recomendado para dar de alta una oficina

5. Preguntas frecuentes

¿Por qué el chatbot dice "aún no está listo"? Ninguna oficina ha subido todavía su PDF.

¿Por qué el mapa no resalta ninguna oficina? La oficina de la respuesta no pertenece al campus que está viendo, o fue eliminada.

¿Puedo usarlo desde el celular? Sí, la interfaz se adapta a pantallas pequeñas.

¿Quién da de alta a los estudiantes? Actualmente se registran directamente en la base de datos por el área de sistemas.

11. Plan de pruebas

Basado en la estructura de ISO/IEC/IEEE 29119-3.

1. Alcance

En alcance	Fuera de alcance
Autenticación y sesión	Calidad lingüística de los modelos de OpenAI
Mapa y resaltado de oficinas	Disponibilidad de OpenStreetMap / CDN
Chatbot (flujo completo y casos de error)	Pruebas de carga a gran escala
Ingesta de PDF	Compatibilidad con navegadores obsoletos (IE)
CRUD de campus, oficinas y administradores	
Controles de seguridad básicos	

2. Estrategia

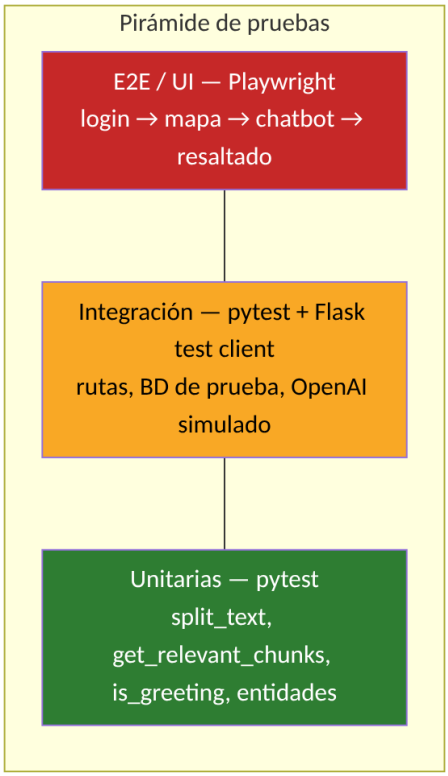


Figura 33. Pirámide de pruebas

Nivel	Herramienta	Qué cubre	Dobles de prueba
Unitario	`pytest`	Funciones puras del motor RAG y entidades	`tiktoken` real; OpenAI simulado con `unittest.mock`

Nivel	Herramienta	Qué cubre	Dobles de prueba
Integración	`pytest` + `app.test_client()`	Rutas, sesiones, CSRF, SQL	MySQL de prueba (contenedor) u objetos simulados; OpenAI simulado
Sistema / E2E	Playwright	Recorridos de usuario en navegador	Backend real con datos semilla
Aceptación	Manual guiada por Gherkin	Historias de usuario (capítulo 3 (Gestión ágil con Scrum))	—
Seguridad	Manual + `bandit`, `pip-audit`	Autorización, CSRF, archivos, dependencias	—

3. Entorno

Elemento	Valor
Base de datos	`lion_solver_test` creada con `docs/database/schema.sql` + datos semilla
Variables	`OPENAI_API_KEY=test` (las llamadas se simulan)
Navegadores	Chromium (escritorio y emulación móvil 390×844)
Datos	1 campus (CUCEI), 3 oficinas, 1 super admin, 1 coordinador, 2 estudiantes (activo/inactivo)

4. Casos de prueba

4.1 Autenticación (RF-01 a RF-06)

ID	Caso	Pasos / datos	Resultado esperado	Prioridad
CP-01	Login estudiante activo	Código y contraseña válidos	302 → `/home`; mapa visible	Alta
CP-02	Login estudiante inactivo	`user_active = 0`	Permanece en login; mensaje de no activo	Alta
CP-03	Login con contraseña incorrecta	Código válido, contraseña errónea	Mensaje de error; sin sesión	Alta
CP-04	Login con código inexistente	Código `999999`	“Usurario no encontrado”	Media
CP-05	Login de administrador	Admin super y regular	302 → `/admin`; panel correspondiente al privilegio	Alta
CP-06	Acceso sin sesión	GET `/home`, `/admin`, POST `/chatbot`	Redirección a `/login`	Alta

4.2 Mapa (RF-07 a RF-10)

ID	Caso	Resultado esperado	Prioridad
CP-07	Carga del mapa	Centro en `campus_lat/lon`, zoom 17, polígono dibujado	Alta
CP-08	Marcadores	Un marcador por oficina del campus; popup con nombre y descripción	Alta
CP-09	Campus sin oficinas	`/home` responde 404 “Campus not found”	Media

ID	Caso	Resultado esperado	Prioridad
CP-10	`highlightOffice(id)`	Icono resaltado, zoom 18, popup abierto; los demás vuelven al icono normal	Alta

4.3 Chatbot (RF-11 a RF-16)

ID	Caso	Datos	Resultado esperado	Prioridad
CP-11	Pregunta válida	"¿Dónde entrego mi servicio social?"	JSON con `reply` no vacío y `office_id`	Alta
CP-12	Índice vacío	Sin `.pkl`	Mensaje "El chatbot aún no está listo…"	Alta
CP-13	Saludo	"hola"	"Hola, ¿en qué puedo ayudarte?" sin llamada a chat completion	Baja
CP-14	Falla del modelo de chat	OpenAI simulado lanza excepción	Mensaje de disculpa, HTTP 200	Media
CP-15	Sin token CSRF	POST sin `X-CSRFToken`	400	Alta
CP-16	Mensaje vacío	Input con espacios	No se envía la petición	Baja
CP-17	Selección de oficina	Fragmentos simulados con similitudes conocidas	`office_id` = oficina del fragmento con mayor similitud	Alta

4.4 Gestión de oficina e ingesta (RF-17 a RF-20)

ID	Caso	Resultado esperado	Prioridad
CP-18	Editar datos de oficina	Valores actualizados en `offices`; mensaje de éxito	Alta
CP-19	Subir PDF con texto	Se crea `embeddings/embeddings_<office_id>.pkl` con *N* registros; índice recargado	Alta
CP-20	Subir PDF escaneado	Mensaje "El PDF no contiene texto…"; no se crea `.pkl`	Media
CP-21	Reemplazo de conocimiento	Segundo PDF sustituye al primero para la misma oficina	Media
CP-22	`split_text`	Ningún fragmento supera 500 tokens; no se pierde texto	Alta

4.5 Administración (RF-21 a RF-23)

ID	Caso	Resultado esperado	Prioridad
CP-23	Alta de campus con GeoJSON válido	Registro creado; aparece en `/campus_view`	Alta
CP-24	Alta de campus con JSON inválido	Mensaje "El archivo JSON no es válido."	Media
CP-25	Edición de campus	Coordenadas y polígono actualizados	Media
CP-26	Baja de campus	Registro eliminado (o error controlado si tiene dependencias)	Media
CP-27	Alta de oficina	Aparece en listado y en el mapa del campus	Alta
CP-28	Edición de ubicación	Marcador en la nueva posición	Media
CP-29	Baja de oficina	Desaparece de listado y mapa	Media
CP-30	Alta de administrador	Contraseña almacenada con hash (no en claro)	Alta

ID	Caso	Resultado esperado	Prioridad
CP-31	Edición de privilegio	Cambia el panel que ve al iniciar sesión	Media
CP-32	Baja de administrador	No puede iniciar sesión	Media

4.6 Seguridad

ID	Caso	Resultado esperado	Estado v4.1
CP-33	Estudiante llama `POST /admin_Add`	403	✘ Falla (SEC-01)
CP-34	Coordinador llama `POST /campus_Delete`	403	✘ Falla (SEC-01)
CP-35	Inyección SQL en `codigo`	Sin efecto (consulta parametrizada)	✔
CP-36	Subir archivo `../../app.py` como PDF	Nombre saneado; sin escritura fuera de `uploads/`	⚠ Revisar (SEC-03)
CP-37	XSS en `office_desc` (`<img onerror>`)	Se muestra como texto	✘ Falla (SEC-04)

5. Ejemplos de pruebas automatizadas

Estructura propuesta:

```
tests/
├── conftest.py          # app de prueba, cliente, mocks de OpenAI
├── test_rag.py          # split_text, get_relevant_chunks, get_answer
├── test_auth.py         # login/logout
└── test_routes.py       # CRUD y autorización
```

`tests/test_rag.py`:

```python
import tiktoken
from unittest.mock import patch

import app as lion


def test_split_text_divide_y_conserva_el_texto():
    enc = tiktoken.get_encoding("cl100k_base")
    oraciones = ["Esta es una oración de prueba número %d" % i for i in range(500)]
    chunks = lion.split_text(". ".join(oraciones), max_tokens=100)
    assert len(chunks) > 1
    assert all(chunks)
    # El límite se mide por oración sin contar el ". " que se agrega al unirlas,
    # por eso se tolera un pequeño excedente (una ficha por oración).
    assert all(len(enc.encode(c)) <= 100 + len(c.split(". ")) for c in chunks)
    unido = " ".join(chunks)
    assert all(o in unido for o in oraciones)


def test_get_relevant_chunks_ordena_por_similitud():
    embeddings = [
        {"embedding": [1.0, 0.0], "text": "A", "office_id": "1"},
        {"embedding": [0.0, 1.0], "text": "B", "office_id": "2"},
    ]
    fake = {"data": [{"embedding": [0.1, 0.9]}]}
    with patch.object(lion.openai.Embedding, "create", return_value=fake):
        top = lion.get_relevant_chunks("pregunta", embeddings, top_k=1)
    assert top[0][1]["office_id"] == "2"
```

```
def test_saludo_no_llama_al_modelo():
    with patch.object(lion.openai.ChatCompletion, "create") as chat:
        assert lion.get_answer("hola", "", {}) == "Hola, ¿en qué puedo ayudarte?"
        chat.assert_not_called()
```

Ejecución (desde la raíz, con `src/` en el *path*):

```
pip install pytest
PYTHONPATH=src pytest -q
```

> Importar `app` ejecuta la carga de embeddings y requiere `src/config.py`. Para pruebas aisladas se recomienda extraer el motor RAG a un módulo propio (`src/rag.py`), ver DT-04.

6. Criterios de entrada y salida

Criterio	Entrada	Salida
Código	Rama con la historia terminada	Todas las pruebas de prioridad Alta pasan
Defectos	—	0 defectos críticos o altos abiertos
Cobertura	—	≥ 70 % en `models/` y motor RAG
Seguridad	—	CP-33 a CP-37 en verde antes de producción

7. Clasificación de defectos

Severidad	Definición	Ejemplo
Crítica	Pérdida de datos o brecha de seguridad	Estudiante puede borrar administradores
Alta	Funcionalidad principal no disponible	El chatbot siempre responde error
Media	Funcionalidad degradada con alternativa	Formulario de edición borra campos vacíos
Baja	Cosmético	Errata “Usurario”

12. Seguridad, riesgos y deuda técnica

Revisión estática del código de la versión 4.1. Severidad según CVSS cualitativo (Crítica · Alta · Media · Baja).

1. Controles existentes

Control	Implementación	Referencia
Hash de contraseñas	`generate_password_hash` / `check_password_hash` (Werkzeug)	`src/app.py:505`, `src/models/entities/*.py`
Protección CSRF	`CSRFProtect` + `csrf_token` en formularios + `X-CSRFToken` en fetch	`src/app.py:662`, `chatbot.js:47`
Consultas parametrizadas	`cursor.execute(sql, params)` en todo el código	`src/app.py`, `src/models/`
Autenticación de vistas	`@login_required` en todas las rutas privadas	`src/app.py`
Secretos fuera del repositorio	`.env` y `config.py` en `.gitignore`	`.gitignore`
Escapado de plantillas	Autoescape de Jinja2 en vistas HTML	`src/template/`

2. Hallazgos de seguridad

ID	Severidad	Hallazgo	Evidencia	Recomendación
SEC-01	**Crítica**	**Falta de autorización por rol.** Todas las rutas de administración solo exigen sesión; un estudiante autenticado puede crear/editar/borrar campus, oficinas y administradores (incluido crearse una cuenta de super admin) y subir PDFs.	`src/app.py:291-561`, `:601`	Decorador `@role_required('super')` / `@role_required('admin')` que compruebe `isinstance(current_user, Admin)` y `current_user.priv`; responder 403.
SEC-02	Alta	**Edición de oficina ajena.** `/office_Edit` confía en el `office_id` oculto del formulario; un coordinador puede editar cualquier oficina cambiando ese valor.	`src/app.py:431`	Usar `current_user.career` en el servidor en lugar del campo del formulario.
SEC-03	Alta	**Nombre de archivo sin sanear.** `os.path.join('uploads', pdf_file.filename)` permite rutas como `../src/app.py` (*path traversal*) y no valida tipo ni tamaño.	`src/app.py:609`	`werkzeug.utils.secure_filename`, validar extensión y *magic bytes* `%PDF`, `MAX_CONTENT_LENGTH`.

ID	Severidad	Hallazgo	Evidencia	Recomendación
SEC-04	Alta	**XSS almacenado en el mapa.** `bindPopup("<b>" + office_name + "</b> " + office_desc)` inserta HTML sin escapar; además `{{ offices_json | safe }}` dentro de `<script>` permite cerrar la etiqueta con `</script>`. Un coordinador podría ejecutar JavaScript en el navegador de todos los estudiantes.	`src/static/js/mapa.js:31`, `src/template/home.html:76`	Construir el popup con nodos DOM / `textContent`; usar el filtro `tojson` de Jinja2 (escapa <, >, &).
SEC-05	Media	**Colisión de identificadores.** `load_user` busca primero en `users` y luego en `admins`; si un código existe en ambas tablas, la sesión de un administrador se recarga como estudiante (o viceversa).	`src/app.py:203-212`	Prefijar el id de sesión (`u:123` / `a:123`) o unificar cuentas en una tabla con rol.
SEC-06	Media	**Deserialización con `pickle`.** Todos los `.pkl` de `embeddings/` se cargan al iniciar; un archivo manipulado permitiría ejecución de código.	`src/app.py:112-120`	Guardar embeddings en JSON/NumPy (`.npy`) o en un almacén vectorial; restringir permisos de la carpeta.
SEC-07	Media	**Inyección de prompt.** El texto de los PDF y la pregunta se insertan sin delimitar en el prompt.	`src/app.py:153-162`	Delimitar contexto (p. ej. etiquetas), mensaje de sistema con instrucciones, filtrar salida.
SEC-08	Media	**Sin límite de frecuencia.** `/login` y `/chatbot` no limitan intentos: fuerza bruta de contraseñas y consumo de la API de OpenAI.	`src/app.py:222`, `:564`	`Flask-Limiter` (p. ej. 5/min en login, 20/min en chatbot por usuario).
SEC-09	Media	**Configuración solo en `__main__`.** Con otro servidor (`flask run`, Gunicorn) no se activan CSRF ni la configuración.	`src/app.py:659-665`	Patrón *application factory* (`create_app()`).
SEC-10	Baja	**Logout por GET.** Permite cerrar la sesión de otro usuario mediante un enlace o imagen (CSRF de logout).	`src/app.py:254`	`POST /logout` con token CSRF.
SEC-11	Baja	**Registro de datos sensibles.** Se imprimen en consola las preguntas y respuestas completas.	`src/app.py:567-597`	Usar `logging` con niveles; no registrar contenido de usuarios en producción.

ID	Severidad	Hallazgo	Evidencia	Recomendación
SEC-12	Baja	**Dependencias sin auditar.** Versiones fijadas de 2024 (`openai 0.28`, `PyPDF2 3.0.1` —obsoleto, sucesor `pypdf`).	`requirements.txt`	`pip-audit` en CI; Dependabot.

2.1 Ejemplo de corrección para SEC-01

```
from functools import wraps
from flask import abort

def role_required(super_only=False):
    def decorator(view):
        @wraps(view)
        def wrapper(*args, **kwargs):
            if not isinstance(current_user, Admin):
                abort(403)
            if super_only and int(current_user.priv) != 1:
                abort(403)
            return view(*args, **kwargs)
        return wrapper
    return decorator


@app.route('/admin_Add', methods=["POST"])
@login_required
@role_required(super_only=True)
def admin_Add():
    ...
```

3. Matriz de riesgos del proyecto

Probabilidad (P) e impacto (I) de 1 a 5; exposición = P × I.

ID	Riesgo	P	I	Exposición	Respuesta	Responsable
R-01	Uso indebido de rutas de administración por usuarios no autorizados (SEC-01)	4	5	**20**	Mitigar: HU-14 en el próximo sprint	Developers
R-02	Retiro de la API `openai<1.0`, `ada-002` o `gpt-3.5-turbo`	4	4	**16**	Mitigar: HU-17	Developers
R-03	Respuestas incorrectas o inventadas del chatbot	3	4	12	Mitigar: prompt restrictivo, evaluación, aviso al usuario	PO + Developers
R-04	Costos imprevistos de OpenAI	3	3	9	Mitigar: límites de gasto y de frecuencia	PO
R-05	PDFs desactualizados en las oficinas	3	3	9	Aceptar/mitigar: recordatorio semestral a coordinadores	PO
R-06	Pérdida de `embeddings/` o `uploads/`	2	3	6	Mitigar: respaldo junto con la BD	Operación
R-07	Dependencia de un solo desarrollador (*bus factor* = 1)	4	4	**16**	Mitigar: esta documentación, revisión de código, pares	Scrum Master

ID	Riesgo	P	I	Exposición	Respuesta	Responsable
R-08	Caída de teselas de OpenStreetMap	2	2	4	Aceptar; alternativa: proveedor de teselas propio	Operación
R-09	Datos personales de estudiantes (nombre, correo) expuestos	2	5	10	Mitigar: HTTPS, mínimos privilegios, aviso de privacidad	PO

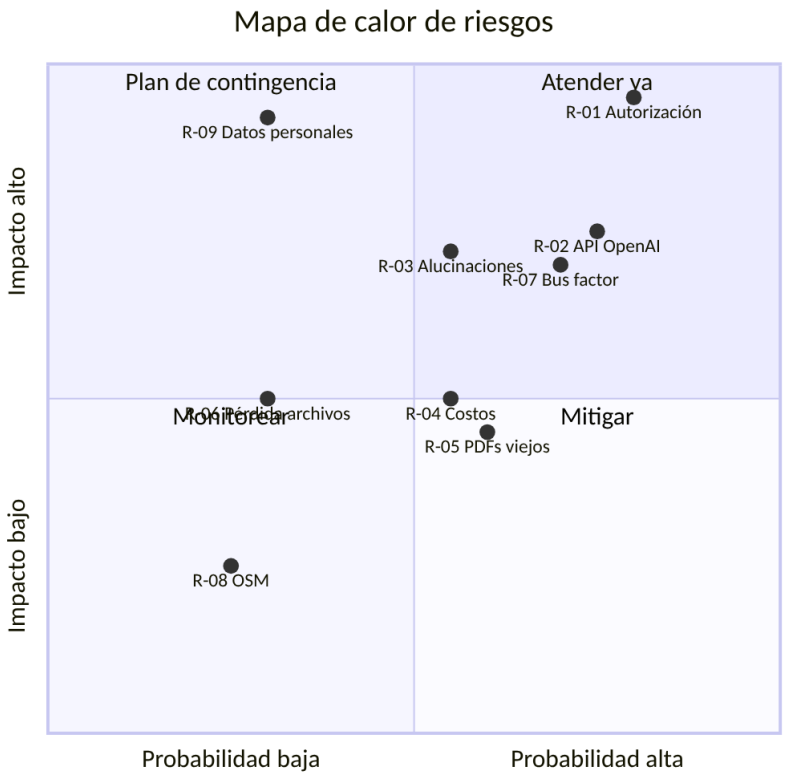


Figura 34. Mapa de calor de riesgos

4. Deuda técnica

ID	Tipo	Descripción	Ubicación	Esfuerzo	Prioridad
DT-01	Dependencias	SDK `openai==0.28` (API heredada `openai.Embedding` / `openai.ChatCompletion`) y modelos antiguos.	`src/app.py:94,125,165`	M	Alta
DT-02	Código muerto	`app2.py` es una copia antigua sin chatbot; `index.html` y `login.js` están vacíos; `render_template_string`, `numpy` sin uso.	`src/`	S	Media

ID	Tipo	Descripción	Ubicación	Esfuerzo	Prioridad
DT-03	Dependencias	`folium`, `branca`, `pandas`, `metrics`, `pairwise`, `cosine-similarity` instalados sin uso tras ADR-005.	`requirements.txt`	S	Baja
DT-04	Diseño	`app.py` concentra rutas, SQL y RAG (665 líneas).	`src/app.py`	L	Media
DT-05	Diseño	El atributo `password` de las entidades almacena texto, booleano o `None` según el momento.	`src/models/entities/`	S	Baja
DT-06	UX / datos	Formularios de edición del coordinador no precargan valores; guardar en blanco borra datos.	`src/template/admin.html`	S	Alta
DT-07	Error	`GET /upload_pdf` y la rama de “PDF sin texto” redirigen a `upload_pdf.html`, que no existe (500).	`src/app.py:616,644`	S	Alta
DT-08	Calidad	Sin pruebas automatizadas ni CI.	—	M	Alta
DT-09	Configuración	Rutas `embeddings/` y `uploads/` relativas al directorio de trabajo; `PORT` se lee pero no se usa en `app.run()`.	`src/app.py:186,607,660`	S	Media
DT-10	Operación	Sin Dockerfile ni punto de entrada WSGI.	—	M	Media
DT-11	Datos	Al borrar una oficina su `.pkl` no se elimina; el chatbot puede seguir sugiriéndola.	`src/app.py:463`	S	Media
DT-12	Consistencia	El comentario dice “office_id más común” pero se usa el top-1; `office_info` no se usa en el prompt.	`src/app.py:585,144`	S	Media
DT-13	Codificación	`requirements.txt` en UTF-16 (algunas herramientas no lo leen).	`requirements.txt`	S	Baja
DT-14	Datos	`campus_coords` se envía al conector MySQL como `dict`; conviene serializarlo explícitamente con `json.dumps` antes del `INSERT`/`UPDATE`.	`src/app.py:315,345`	S	Media

ID	Tipo	Descripción	Ubicación	Esfuerzo	Prioridad
DT-15	Mensajes	`flash(mensaje, e)` pasa la excepción como *categoría*; erratas como “Usurario” o “Cuenta editada” en edición de oficina.	`src/app.py:248,321,441`	S	Baja
DT-16	Consultas	Uso extendido de `SELECT *` con `JOIN`, que mezcla columnas y expone campos innecesarios (p. ej. hashes) a las plantillas.	`src/app.py:399,482,554`	S	Media

Esfuerzo: S ≤ ½ día · M 1–3 días · L > 3 días.

5. Plan de remediación

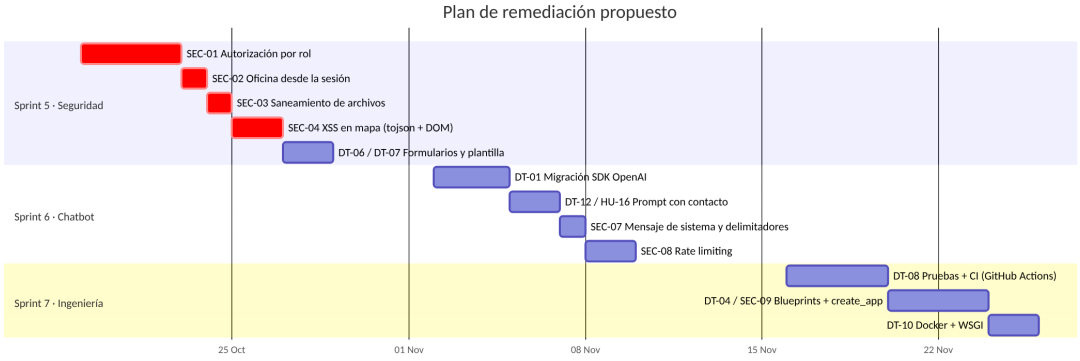


Figura 35. Plan de remediación

Las fechas son una propuesta a partir del siguiente Sprint Planning y deben ajustarse a la capacidad real del equipo.

Anexo A. Historial de cambios

Todos los cambios relevantes del proyecto se documentan en este archivo.

El formato sigue Keep a Changelog y el proyecto usa versionado semántico a partir de la siguiente versión. Las entradas hasta 4.1 se reconstruyeron del historial de Git.

Versión sin publicar

Añadido

- Documentación completa en `docs/`: visión, requerimientos, Scrum, arquitectura (C4), diagramas UML, base de datos, rutas HTTP, chatbot RAG, instalación, manual de usuario, plan de pruebas, y seguridad/riesgos/deuda técnica.
- Script de base de datos reconstruido `docs/database/schema.sql`.
- Plantillas de configuración `.env.example` y `src/config.example.py`.
- `CONTRIBUTING.md` y este `CHANGELOG.md`.

Versión 4.1 - 2024-12-12

Cambiado

- Limpieza de `chatbot.js` (se eliminan dos líneas de depuración).

Versión 4.0 - 2024-11-21

Añadido

- `office_id` incluido en la información de oficina devuelta por `get_office_info()`.
- Registro en consola de la similitud de cada fragmento recuperado.

Cambiado

- La oficina asociada a la respuesta pasa a ser la del fragmento **más similar** (top-1) en lugar de la más frecuente entre los 5 recuperados.

Versión 3.0 - 2024-11-21

Añadido

- Mapa en el cliente con **Leaflet** (`mapa.js`) y nuevos iconos de marcador normal y resaltado.
- Función `highlightOffice()`: el chatbot centra y resalta en el mapa la oficina de la respuesta.
- Estilos `mapa.css`.

Cambiado

- Se sustituye el mapa generado en servidor con **Folium** (iframe) por datos JSON renderizados en el navegador.

- La consulta del mapa ya no exige que cada oficina tenga un administrador asignado.

Versión 2.0 - 2024-10-07

Añadido

- Carga de PDFs a la carpeta `uploads/` y primer documento de la Unidad de Servicio Social.
- Mensajes de progreso durante la generación de embeddings.

Cambiado

- `split_text()` divide por palabras las oraciones que exceden el límite de tokens.
- Mejoras de estilo en la ventana del chatbot.

Versión 1.2 - 2024-10-07

Añadido

- CRUD de oficinas (`office_view.html`, `/office_View`, `/office_Add`, `/office_Edit_Rec`, `/office_Delete`).
- Tarjeta “Ver Oficinas” en el panel de super administrador.

Versión 1.1 - 2024-10-06

Añadido

- Aplicación Flask con autenticación de estudiantes y administradores (Flask-Login).
- Mapa del campus con Folium y GeoJSON (`cucei.json`).
- CRUD de campus y administradores.
- Panel de coordinador: edición de oficina y entrenamiento del chatbot.
- Chatbot RAG con OpenAI (embeddings `text-embedding-ada-002`, respuestas `gpt-3.5-turbo`).
- Protección CSRF con Flask-WTF.

Versión 1.0 - 2024-10-06

Añadido

- Repositorio inicial y README.

Anexo B. Guía de contribución

¡Gracias por contribuir a Lion Solver! Esta guía resume el flujo de trabajo del equipo. El marco de gestión completo está en capítulo 3 (Gestión ágil con Scrum).

Flujo de trabajo

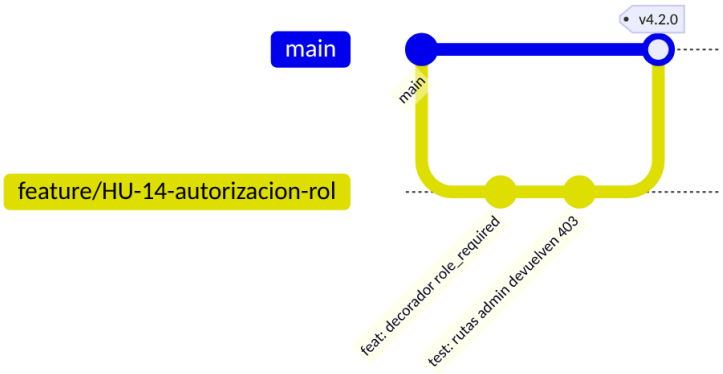


Figura 36. Flujo de trabajo con Git

1. Toma una historia del *Sprint Backlog* (debe cumplir la Definition of Ready).
2. Crea una rama desde `main`:

Tipo	Formato	Ejemplo
Historia	`feature/HU-xx-descripcion`	`feature/HU-14-autorizacion-rol`
Defecto	`fix/descripcion`	`fix/upload-pdf-template`
Deuda técnica	`refactor/DT-xx-descripcion`	`refactor/DT-04-blueprints`
Documentación	`docs/descripcion`	`docs/manual-coordinador`

3. Haz commits pequeños con Conventional Commits:

```
feat(chatbot): incluir horario de la oficina en el prompt
fix(admin): precargar valores en el formulario de edición
docs(uml): actualizar diagrama de secuencia del login
```

4. Abre un *pull request* hacia `main` con:
   - referencia a la historia (`HU-xx`) o hallazgo (`SEC-xx`, `DT-xx`);
   - descripción del cambio y cómo probarlo;
   - capturas si cambia la interfaz.
5. Al menos una revisión aprobada y la **Definition of Done** cumplida antes de fusionar.

Estándares de código

Ámbito	Norma
Python	PEP 8; nombres de funciones en `snake_case`; consultas SQL siempre parametrizadas

Ámbito	Norma
Rutas nuevas	`@login_required` y verificación de rol cuando corresponda
Plantillas	Heredan de `layout.html`; todo formulario `POST` incluye `{{ csrf_token() }}`
JavaScript	Sin `innerHTML` con datos de usuario; usar `textContent`
Datos a JS	Usar el filtro `tojson` en lugar de `| safe`
Secretos	Nunca en el código; usar `.env` / `config.py`

Pruebas

```
PYTHONPATH=src pytest -q
```

Toda lógica nueva debe acompañarse de pruebas (ver capítulo 11 (Plan de pruebas)).

Documentación

Si tu cambio afecta comportamiento, rutas, base de datos o arquitectura, actualiza el documento correspondiente en `docs/` y agrega una entrada en anexo A (Historial de cambios) bajo **[Sin publicar]**.

Anexo C. Script de base de datos

Contenido del archivo `docs/database/schema.sql`, reconstruido a partir de las consultas de la aplicación (versión 4.1).

```sql
-- ========================================================================
--  Lion Solver — Esquema de base de datos (MySQL 8 / MariaDB 10.5+)
-- ------------------------------------------------------------------------
--  IMPORTANTE: este esquema se RECONSTRUYÓ a partir de las consultas
--  SQL de src/app.py y src/models/*.py (versión 4.1). El repositorio
--  original no incluía el DDL. Revise tipos y longitudes contra la base
--  de datos productiva antes de usarlo para migrar datos reales.
-- ========================================================================

CREATE DATABASE IF NOT EXISTS lion_solver
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE lion_solver;

-- ------------------------------------------------------------------------
-- campus: centro universitario con su polígono GeoJSON
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS campus (
    campus_name    VARCHAR(100)   NOT NULL,
    campus_coords  JSON           NOT NULL COMMENT 'GeoJSON FeatureCollection del
perímetro',
    campus_lat     DECIMAL(10, 7) NOT NULL COMMENT 'Latitud del centro del mapa',
    campus_lon     DECIMAL(10, 7) NOT NULL COMMENT 'Longitud del centro del mapa',
    CONSTRAINT pk_campus PRIMARY KEY (campus_name)
) ENGINE = InnoDB;

-- ------------------------------------------------------------------------
-- offices: oficinas/dependencias dentro de un campus
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS offices (
    office_id      INT            NOT NULL AUTO_INCREMENT,
    office_campus  VARCHAR(100)   NOT NULL,
    office_name    VARCHAR(150)   NOT NULL,
    office_desc    TEXT               NULL,
    office_phone   VARCHAR(50)        NULL,
    office_hours   VARCHAR(100)       NULL,
    office_lat     DECIMAL(10, 7) NOT NULL,
    office_lon     DECIMAL(10, 7) NOT NULL,
    office_career  VARCHAR(150)       NULL COMMENT 'Carrera o área que atiende',
    CONSTRAINT pk_offices PRIMARY KEY (office_id),
    CONSTRAINT fk_offices_campus FOREIGN KEY (office_campus)
        REFERENCES campus (campus_name)
        ON UPDATE CASCADE
        ON DELETE CASCADE
) ENGINE = InnoDB;

CREATE INDEX ix_offices_campus ON offices (office_campus);

-- ------------------------------------------------------------------------
-- admins: coordinadores de oficina (priv = 0) y super admins (priv = 1)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS admins (
    admin_cod       INT           NOT NULL COMMENT 'Código institucional (usuario de
login)',
    admin_password  VARCHAR(255)  NOT NULL COMMENT 'Hash Werkzeug (scrypt/pbkdf2)',
    admin_campus    VARCHAR(100)  NOT NULL,
```

```
    admin_office    INT              NULL,
    admin_fullname  VARCHAR(150)  NOT NULL,
    admin_email     VARCHAR(150)  NOT NULL,
    admin_priv      TINYINT(1)    NOT NULL DEFAULT 0 COMMENT '0 = coordinador, 1 = super
admin',
    CONSTRAINT pk_admins PRIMARY KEY (admin_cod),
    CONSTRAINT fk_admins_campus FOREIGN KEY (admin_campus)
        REFERENCES campus (campus_name)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT fk_admins_office FOREIGN KEY (admin_office)
        REFERENCES offices (office_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL
) ENGINE = InnoDB;

-- ---------------------------------------------------------------------
-- users: estudiantes
-- ---------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS users (
    user_cod       INT           NOT NULL COMMENT 'Código de estudiante (usuario de
login)',
    user_password  VARCHAR(255)  NOT NULL COMMENT 'Hash Werkzeug',
    user_campus    VARCHAR(100)  NOT NULL,
    user_fullname  VARCHAR(150)  NOT NULL,
    user_email     VARCHAR(150)  NOT NULL,
    user_career    VARCHAR(150)      NULL,
    user_active    TINYINT(1)    NOT NULL DEFAULT 1 COMMENT '1 = puede iniciar sesión',
    CONSTRAINT pk_users PRIMARY KEY (user_cod),
    CONSTRAINT fk_users_campus FOREIGN KEY (user_campus)
        REFERENCES campus (campus_name)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
) ENGINE = InnoDB;

-- =====================================================================
--  Datos semilla (opcional)
--  Las contraseñas deben generarse con werkzeug.security.generate_password_hash:
--    python -c "from werkzeug.security import generate_password_hash as g;
print(g('CambiaEsto123'))"
--  y pegar el resultado en lugar de <HASH>.
-- =====================================================================
-- INSERT INTO campus (campus_name, campus_coords, campus_lat, campus_lon)
-- VALUES ('CUCEI', '<contenido de src/static/json/cucei.json>', 20.6567, -103.3254);
--
-- INSERT INTO offices (office_campus, office_name, office_desc, office_phone,
office_hours, office_lat, office_lon, office_career)
-- VALUES ('CUCEI', 'Unidad de Servicio Social', 'Trámites de servicio social', '33 0000
0000', 'L-V 9:00-15:00', 20.6560, -103.3260, 'General');
--
-- INSERT INTO admins (admin_cod, admin_password, admin_campus, admin_office,
admin_fullname, admin_email, admin_priv)
-- VALUES (1000, '<HASH>', 'CUCEI', 1, 'Super Administrador', 'admin@example.com', 1);
--
-- INSERT INTO users (user_cod, user_password, user_campus, user_fullname, user_email,
user_career, user_active)
-- VALUES (219000001, '<HASH>', 'CUCEI', 'Estudiante de Prueba',
'estudiante@example.com', 'INCO', 1);
```